

Deep learning of first-order nonlinear hyperbolic conservation law solvers

Victor Morand ^{a,b,*}, Nils Müller ^{c,d}, Ryan Weightman ^e, Benedetto Piccoli ^e,
Alexander Keimer ^f, Alexandre M. Bayen ^b

^a Ecole Polytechnique, Palaiseau, France

^b Department of Electrical Engineering and Computer Science, UC Berkeley, CA, United States of America

^c Max Planck Institute for Software Systems, Saarland Informatics Campus, Germany

^d Max Planck Institute for Intelligent Systems, Germany

^e Department of Mathematics, Rutgers University, NJ, United States of America

^f Department of Mathematics, FAU Erlangen-Nürnberg, Germany

ARTICLE INFO

Keywords:

Finite volume methods

Numerical schemes

Neural networks

Approximation of numerical schemes for conservation laws by neural nets

Approximation of numerical schemes for conservation laws by polynomials

ABSTRACT

In this contribution, we study the numerical approximation of scalar conservation laws by computational optimization of the numerical flux function in a first-order finite volume method. The cell-averaging inherent to finite volume schemes presents a challenge in the design of such numerical flux functions toward achieving high solution accuracy. Dense neural networks, as well as (piecewise) polynomials, serve as function classes for the numerical flux. Using a parametrization of such a function class, we optimize the numerical flux with respect to its solution accuracy on a set of Riemann problems for which we have closed-form solutions. By design, our resulting first-order finite volume schemes are conservative.

We show empirically that the proposed schemes can achieve higher accuracy than the Godunov and Enquist-Osher method on a wide range of discretizations.

The proposed method can be applied to the inverse problem, i.e., the data-driven synthesis of macroscopic physical laws for real-world flux phenomena.

1. Introduction

Conservation laws play a crucial role in many physical laws (conservation of mass, energy, momentum) as well as in macroscopic descriptions of particle phenomena (traffic, herds of animals, etc.). In physics, the identification of a conservation quantity is often a crucial step in understanding complex physical dynamics. Also, the mathematical theory of conservation laws, which can roughly be characterized as a class of hyperbolic PDEs (partial differential equations), has been intensively studied as it shows interesting behavior such as the emergence of discontinuities for arbitrary smooth initial datum in arbitrary small time and more. This is why the present work studies the numerical approximation of first-order hyperbolic conservation laws of the form

$$\partial_t \rho(t, x) + \partial_x (f(\rho(t, x))) = 0 \quad (1)$$

* Corresponding author.

E-mail address: victor.morand@polytechnique.edu (V. Morand).

<https://doi.org/10.1016/j.jcp.2024.113114>

Received 30 September 2023; Received in revised form 27 March 2024; Accepted 13 May 2024

Available online 17 May 2024

0021-9991/© 2024 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

with a proper initial datum $\rho(0, x) = \rho_0(x)$, $x \in \mathbb{R}$, $\rho_0 \in L^\infty(\mathbb{R})$ and $f : \mathbb{R} \rightarrow \mathbb{R}$ locally Lipschitz-continuous and strictly concave (the concavity choice is due to the particular interest of modeling traffic flow, where the flux is supposed to be concave and not convex, however, all the obtained results can easily be applied to strictly convex fluxes as well). We are interested in the approximation of weak entropy solutions by numerical methods and investigating how we can optimize the emerging “numerical flux functions” within a given function class to improve on the accuracy of existing numerical solvers. An approximation by neural networks as well as by products of monotonous piecewise polynomials turns out to improve when refining the discretization grid. We demonstrate this with a variety of numerical examples.

1.1. Fundamentals of conservation laws and numerical schemes

Conservation laws appear as fundamental models of physical laws in many systems, such as in gas dynamics, traffic, and classical mechanics. The theory developed from the 50s of the 20th century for the scalar case all the way to the early 2000s for the systems case, both in one spatial dimension. For a thorough overview concerning conservation laws, we refer the reader to the books [1–6] and the references therein. In these contributions, the foundations of conservation laws are laid, ranging from viscosity approximations of conservation laws to the definition of entropy solutions and entropy admissibility conditions (see in particular also [7,8]). Some of them also consider systems of conservation laws, numerical approximations (see in particular [9]), and wavefront tracking (see [10]).

As most conservation laws do not exhibit explicit solution formulae, numerical methods are used to approximate solutions numerically. Additionally, one aims for methods that are shown to converge if the discretization in space and time becomes arbitrarily fine. As the aim of this contribution is to find good numerical approximations of solutions by optimizing the underlying numerical flux functions obtained from finite volume schemes, we will use some of the available algorithms with the disclaimer that the used methods are by no means exhaustive. We refer the reader to Section 3, where we investigate the used solvers in greater detail to have benchmarks to compare our obtained methods to. However, higher-order schemes like flux/slope-limiter [11] (W)ENO (weighted essentially non-oscillatory) [12–14], MOOD (Multi-dimensional Optimal Order Detection) [15] and discontinuous Galerkin [16,17] would be other methods of comparison one could consider in future work.

1.2. Neural networks and their applications in (numerical approximations of) PDEs

A neural network is a mathematical function that maps input data to an output through a composition of non-linear functions called *layers*, made of processing units called *neurons*. The neurons perform a simple mathematical operation on their layer’s input data, typically an affine function of input values followed by a nonlinear function called *activation function* which is applied element-wise.

Neural networks have been proven to be *universal approximators* in 1989 [18], i.e., for any given positive precision and continuous function on a compact set, there exists a neural network that L^∞ -approximates the function within the precision. They initially did not achieve the widespread success seen today before the discovery and use of reverse-mode automatic differentiation to efficiently compute gradients, and until large-scale and parallel computing resources to optimize their parameters became widely available [19]. Their capacity to approximate, along with the immense amount of scientific resources invested by the machine-learning community to efficiently solve the highly multi-modal problem of optimizing their parameters [19] makes them a promising choice in our attempt to learn new numerical schemes.

The recent development of deep learning frameworks allowed some studies to solve PDEs with neural networks. More precisely, one can minimize the residual of a solution candidate to a PDE that is a neural network over its parameters. Such models are called *Physics Informed Neural Networks* or PINNs [20]. In traffic estimation and forecasting using neural networks, this problem formulation is most widely used [21]. The main limitation of PINNs is that the neural network is the solutions itself, such that the error-minimization has to be performed for each solution to be computed: This is computationally expensive and does not certify convergence to the real physical solution. Other approaches try to solve specific conservation laws by harnessing the predictive capabilities of *Recurrent Neural Networks* (RNNs), especially *Long Short-Term Memory* or LSTMs [22–24] that are stateful mathematical models/dynamical systems, i.e., they allow to retain information through time.

In [25], the authors provide estimations on the generalization error of physics-informed neural networks, requiring stability estimates of the nonlinear PDEs considered. In [26], the authors use Kruzkov’s Entropy formulation to rewrite the underlying optimization problem as a min-max problem (minimizing the integral term when maximizing over “all” Kruzkov entropies and all test functions). The advantage of the integral formulation lies in the fact that solutions are not “matched” point-wise but in an integral (“weak”) sense, so that no higher regularity on the solutions is required (which is particularly interesting for conservation laws). Even more, one does not require a training set. Similarly, in [27] a deep learning methodology is proposed that can learn the weak solutions of PDEs via a min-max formulation, which transforms the PDE problem into a min-max optimization problem to identify weak solutions, providing reasonably good estimates of solutions on domains where conventional numerical methods may be tedious or hard to apply. In [28] the authors show when approximating the solution of parametric hyperbolic scalar conservation laws with ReLU (Rectified Linear Unit, compare also later Fig. 6) neural networks, the error can be made as small as desired, and such formulations overcome the curse of dimensionality. The work of [29] focuses on the development of a physics-informed deep learning framework to approximate solutions to nonlinear PDEs that can develop shocks or discontinuities without any a-priori knowledge of the solution or the location of the discontinuities. The authors of [30] take an approach similar to that of this contribution by

optimizing the numerical flux. However, they do not “train” on Riemann data only but on data consisting of “exact mapping between cell averages and interpolated values for a set of integrable functions.”

While the present work describes the design of numerical solvers for conservation laws, [31] considers particular techniques for improving the approximation of single solutions using neural networks. The authors of [32] compare different methods in which neural networks parameterize a space containing an approximate solution to conservation laws describing gas transport. The solution to the conservation law is then found by solving the multi-objective optimization problem of minimizing the sum of the L^2 -metric distances to specified initial and boundary data as well as the L^2 -norm of the residual of the solution over the parameters of the neural network. While our work describes the design of numerical solvers for conservation laws, this work considers as the previous one the approximation of single solutions using neural networks. Also, [33] considers the approximation of particular solutions using neural networks. [34] use a Lagrangian formulation of a conservative system to turn conserved quantities into symmetries of solutions using Noether’s theorem. These symmetries are enforced on a neural network that models the Lagrangian of the system. In contrast to the present work, the method is applied to equations of particle motion and w.r.t. physically different problem formulations. In [35], the authors use the physical property of conservation of mass to modify PINN (physically informed neural networks) using control volume discretization (cv-PINN) (variational physics-informed neural networks) and [36] use solutions to Riemann problems for systems of conservation laws to train a neural network. The trained network is then used in a finite volume solver to find solutions to Riemann problems with three convincing examples of systems including the Euler equation. The main difference with our approach is their focus on the Riemann problem only, instead of Cauchy problems, but dealing with the systems case. [37] modify the PINN approach taking advantage of natural symmetries of the equations (SPINN) (Sparse, Physics-based, and partially interpretable Neural Networks). The trained network is proved to provide superior results w.r.t. to PINN in case of strong symmetries. Specific applications of PINNs with additional diffusion in both 1D and 2D have also been done on Riemann problems in [38,39] and are covered more generally in [40].

Alongside PINNs, significant effort has been made to incorporate neural networks inside known numerical methods. The framework of *Neural Operators*[41] describes how one can learn a broad class of numerical operators with neural networks, with applications in learning various solution operators for PDEs. [42] successfully applies this idea to flux limiters, [43] does the same to learn an artificial viscosity model with neural networks. More generally, it is also possible to learn the whole numerical scheme with a neural network, as done in [44]. The range of possible schemes is then huge, and the learned scheme is a black-box with no guarantee for convergence nor mass-conservation. This work goes in the same direction, while focusing on conservation laws. Our framework allows to learn a huge class of first-order conservative numerical schemes, while conserving the interpretability and the nice properties that our added conservative structure brings. In [45] the authors add an artificial viscosity to the PDE whenever the numerical solver produces unwanted oscillations. They formulate this by means of an optimal control problem where the control is the artificial viscosity and the objective function is the fitting to a reference solution. The numerical method used is a discontinuous Galerkin method, and the neural network learns the required artificial viscosity. In [46,47] a method involving neural network-based approximations of discretized differential operators is developed. In contrast to our work, their solver uses the flux function of the physical law, the numerical flux is not modeled explicitly, and the data is not restricted to Riemann solutions. However, the authors improve existing solvers for similar problems and such that exploit similar structure. [48] develops neural network-based correctors for existing spectral methods that apply to smooth problems. The work differs also in its need for non-Riemann data. [49] also augments existing solvers with a neural network. However, in this work it is used as a troubled-cell indicator for high-resolution schemes. The training data also includes non-Riemann data. [50] employs a similar approach with refined detectors in conjunction with discontinuous Galerkin methods.

In the present manuscript, we will only focus on scalar conservation laws. However, we believe that some of the later applied methods could successfully be applied to the other classes of hyperbolic conservation laws as well.

1.3. Organization of the manuscript

As we aim for the optimization of a numerical flux function in our discrete schemes, we will first describe how to derive a precise training and testing set with solutions that are evaluated numerically within the tolerances of machine precision. We will introduce in Section 2 explicit and semi-explicit solutions to Riemann problems and we discuss Hamilton-Jacobi equations which allow under specific assumptions on the flux to compute solutions up to machine precision by solving an unconstrained optimization problem at each space-time point considered. In Section 3, we will then introduce the class of numerical algorithms considered, ranging from Godunov’s method to the Lax-Friedrichs and Enquist-Osher method. These will serve later for benchmarking purposes with our obtained solvers. This is also the section, where we instantiate our function classes (usually based on a so-called three-point stencil) which we will serve as models for the numerical flux: *Specific neural nets and particularly simple piecewise polynomials*.

Section 4 then instantiates the training data, i.e. explicit or semi-explicit solutions, which were introduced in Section 2. This data will serve us in optimizing our numerical flux functions enabling the finite-volume scheme to fit the time and space-dependent data as well as possible and thus requires us to define suitable objective functions (typically, we will go for the ℓ^2 -norm in space-time) that we will optimize. In Section 5, we will discuss our obtained results, validate them against well-known classical methods, and explain and interpret the results. This section is the core of the manuscript, as it demonstrates the capabilities of neural nets but also classical function classes to model accurate solvers to conservation laws. Eventually, in Section 6, we will discuss shortcomings of our chosen approaches, possible remedies and improvements, and open problems to investigate in future work.

2. Explicit and “semi-explicit” solutions

We start with solutions to Riemann problems, i.e., problems where the initial datum is piecewise constant with a single jump discontinuity. They will later serve as generators for our “training data.”

2.1. Riemann problems

Lemma 2.1. Let $\rho_0 \in L^\infty(\mathbb{R})$ be given, assume that f is strictly concave and $\exists \rho_l, \rho_r \in \mathbb{R}$

$$\rho_0(x) = \begin{cases} \rho_l & \text{if } x < 0 \\ \rho_r & \text{if } x > 0. \end{cases} \quad (2)$$

Then, the entropy solution of the conservation law in Eq. (1) can be stated for $\rho_l \leq \rho_r$ (the shock-wave case) as

$$\rho(t, x) = \begin{cases} \rho_l & \text{if } x < \frac{f(\rho_l) - f(\rho_r)}{\rho_l - \rho_r} t \\ \rho_r & \text{if } x > \frac{f(\rho_l) - f(\rho_r)}{\rho_l - \rho_r} t. \end{cases}$$

In the case of $\rho_l \geq \rho_r$ (the rarefaction wave case) we have as the entropy solution

$$\rho(t, x) = \begin{cases} \rho_l & \text{if } x < f'(\rho_l)t, \\ (f')^{-1}\left(\frac{x}{t}\right) & \text{if } f'(\rho_l)t < x < f'(\rho_r)t, \\ \rho_r & \text{if } x > f'(\rho_r)t, \end{cases} \quad (3)$$

where $(f')^{-1}$ is the inverse function to f' which exists due to the strict monotonicity of f' .

Proof. Justification can be found in [51, Section 2.1.3]. $\square$

2.2. Lax-Hopf formula

Riemann problems are fundamental solutions for a rather simplistic initial datum (piecewise constant with one jump discontinuity). The Lax-Hopf formula does not have this restriction and can be applied to all initial data in the class $L^\infty(\mathbb{R})$. It takes advantage of the relation between conservation laws and Hamilton-Jacobi equations which enables it to write down solutions to the conservation law in terms of an unconstrained minimization problem and a differential operator.

Lemma 2.2 (Lax-Hopf formula). Let f be strictly concave, $\lim_{x \rightarrow \pm\infty} \frac{f(x)}{|x|} = -\infty$ and $\rho_0 \in L^\infty(\mathbb{R})$. Define $v_0 \in W_{loc}^{1,\infty}(\mathbb{R})$ as the antiderivative of ρ_0 , i.e.

$$v_0(x) = \int_0^x \rho_0(y) dy, \quad x \in \mathbb{R}.$$

Then, the entropy solution of the conservation law in Eq. (1) with initial datum ρ_0 is given as the spatial derivative of the function

$$v(t, x) := \max_{y \in \mathbb{R}} \left(t \cdot f^*\left(\frac{x-y}{t}\right) + v_0(y) \right), \quad (t, x) \in (0, T) \times \mathbb{R},$$

i.e.,

$$\rho(t, x) = \partial_x v(t, x), \quad \forall (t, x) \in (0, T) \times \mathbb{R}.$$

Thereby, f^* is the convex dual of f , i.e.

$$f^* : \begin{cases} D & \rightarrow \mathbb{R} \\ x & \mapsto \sup_{y \in \mathbb{R}} \{y \cdot x + f(y)\}, \end{cases}$$

for $D := \{x \in \mathbb{R} : \sup_{y \in \mathbb{R}} \{y \cdot x + f(y)\} < \infty\}$.

Proof. See for instance [1, Chapter 3.3]. $\square$

The Lax-Hopf formula is a good tool for us to generate both training and validation sets up to machine precision.

2.3. Some explicit solution used for illustration and benchmarking

Lemma 2.3 (Explicit solutions for some specific initial data). Consider the following Cauchy problem

$$\begin{aligned} \rho_t(t, x) + \left(\rho(t, x) \left(1 - \frac{\rho(t, x)}{4} \right) \right)_x &= 0 & (t, x) \in (0, T) \times \mathbb{R} \\ \rho(0, x) &= \rho_0(x) & x \in \mathbb{R} \end{aligned}$$

and initial datum

$$\rho_0(x) = \begin{cases} 1 & \text{if } x \leq 0 \\ 1 + x & \text{if } x \in (0, 1] \\ 2 & \text{if } x \in (1, 10] \\ 2 + 2(x - 10) & \text{if } x \in (10, 11] \\ 4 & \text{if } x \in (11, 20] \\ 1 & \text{if } x > 20. \end{cases}$$

Then, the solution can be posed as (for the details see [52])

$$\rho(t, x) = \begin{cases} 1 & \text{if } \begin{cases} x > 20 + \frac{1}{2}t \\ \vee \left(t \leq 2 \wedge x \leq \frac{1}{2}t \right) \\ \vee \left(2 \leq t \leq \frac{40}{3} \wedge x \leq \frac{t+2}{4} \right) \\ \vee \left(\frac{40}{3} \leq t \leq \frac{154}{9} \wedge x \leq \frac{43}{6} - \frac{t}{4} \right) \\ \vee \left(t \geq \frac{154}{9} \wedge x \leq \frac{t}{2} - \sqrt{\frac{77}{2}}t + 20 \right) \end{cases} \\ 1 + \frac{2x-t}{2-t} & \text{if } \left(t \leq 2 \wedge \frac{t}{2} \leq x \leq 1 \right) \\ 2 + \frac{2x-20}{1-t} & \text{if } (t \leq 1 \wedge 10 \leq x \leq 11-t) \\ 2 & \text{if } \begin{cases} (t \leq 1 \wedge 1 \leq x \leq 10) \\ \vee \left(1 \leq t \leq 2 \wedge 1 \leq x \leq \frac{21}{2} - \frac{t}{2} \right) \\ \vee \left(2 \leq t \leq \frac{40}{3} \wedge \frac{t+2}{4} \leq x \leq \frac{21-t}{2} \right) \end{cases} \\ 4 & \text{if } \begin{cases} (t \leq 2 \wedge 11-t \leq x \leq 20-t) \\ \vee \left(1 \leq t \leq \frac{40}{3} \wedge \frac{21-t}{2} \leq x \leq 20-t \right) \\ \vee \left(\frac{40}{3} \leq t \leq \frac{154}{9} \wedge \frac{43}{6} - \frac{t}{4} \leq x \leq 20-t \right) \\ \vee (t \leq 1 \wedge t \leq 20-x \wedge t \geq 11-x) \end{cases} \\ 2 - \frac{2x-40}{t} & \text{if } \begin{cases} \left(t \leq \frac{154}{9} \wedge 20-t \leq x \leq \frac{t}{2} + 20 \right) \\ \vee \left(t \geq \frac{154}{9} \wedge \frac{t}{2} - \sqrt{\frac{77}{2}}t + 20 \leq x \leq \frac{t}{2} + 20 \right) \end{cases} \end{cases} \quad (4)$$

The solution stated in Eq. (4) is illustrated later in Fig. 9 and used in Section 5.1.2 to analyze the obtained numerical schemes. Next, we will provide several flux functions that are reasonable in traffic flow modeling and satisfy the claimed assumptions on f .

Remark 1 (Some examples for flux functions in traffic flow modelling). In traffic flow modeling, the flux function f is usually a product of a velocity and the density, i.e.,

$$f(\rho) = \rho V(\rho), \quad \rho \in \mathbb{R}_{\geq 0}$$

with the velocity V yet to be stated. Choices for V could be — assuming that we have nonnegative initial datum — for $\rho_{\max} \in \mathbb{R}_{>0} \ni v_{\max}$

Greenshields [53]: $V_g(\rho) = v_{\max} \left(1 - \frac{\rho}{\rho_{\max}} \right)$, $\rho \in [0, \rho_{\max}]$

Generalized Greenshields [51]: $V_{gg}(\rho) = v_{\max} \left(1 - \left(\frac{\rho}{\rho_{\max}} \right)^n \right)$, $\rho \in [0, \rho_{\max}]$ for $n \in \mathbb{N}_{\geq 1}$

Underwood [51]: $V_u(\rho) = v_{\max} \exp \left(- \frac{\rho}{\rho_{\max}} \right)$, $\rho \in [0, \rho_{\max}]$

Piecewise linear [51]: $V_{\text{pl}}(\rho) = \begin{cases} v_{\max} & \rho \in [0, \rho_c] \\ \frac{v_{\max} \rho_{\max}}{\rho_{\max} - \rho_c} - \rho \frac{v_{\max}}{\rho_{\max} - \rho_c} & \rho \in [\rho_c, \rho_{\max}] \end{cases}$
 with $\rho_c \in (0, \rho_{\max})$ be given.

Except for the piecewise linear case for which the value ρ_c is required to define the velocity, we define it for the remaining cases as

$$\rho_c^i := \arg \max_{\rho \in [0, \rho_{\max}]} \rho \cdot V_i(\rho), \quad i \in \{g, gg, u\}. \quad (5)$$

It is also worth mentioning that the flux functions

$$f_i(\rho) = \rho V_i(\rho), \quad \rho \in [0, \rho_{\max}], \quad i \in \{g, gg, u, \text{pl}\}$$

satisfy the required strict concavity property. For more flux functions used in traffic flow modeling we refer the reader to [6,51].

Having established the required explicit solution formulae as well as some semi-explicit solutions by means of the Lax-Hopf formula, we now turn our attention to numerical algorithms for conservation laws.

3. Numerical algorithms

The literature concerning numerical approximations is vast. We will introduce in the following a class of Lax-Friedrichs' methods in Section 3.2.1, the Godunov scheme in Section 3.2.2 and an Enquist-Osher scheme in Section 3.2.3. We embed all solvers into the general framework of numerical flux functions. We then discuss in Section 3.3 the approximation of solutions via numerical fluxes generated by neural nets and in Section 3.4 the approximation of solutions by means of numerical fluxes generated by piecewise polynomial functions.

In the following, we will tackle numerical approximations of the following Cauchy problem on the time horizon $(0, T)$, $T \in \mathbb{R}_{>0}$ and $f \in W_{\text{loc}}^{1,\infty}(\mathbb{R})$

$$\begin{aligned} \partial_t \rho(t, x) + \partial_x f(\rho(t, x)) &= 0, & (t, x) \in (0, T) \times \mathbb{R} \\ \rho(0, x) &= \rho_0(x), & x \in \mathbb{R}. \end{aligned} \quad (6)$$

3.1. General numerical flux

Given $\delta_t \in (0, T)$ and $\delta_x \in \mathbb{R}_{>0}$, we define for a given spatial length $L \in \mathbb{R}_{>0}$, $\mathbb{N}_{\geq 1} \ni N_x := \lceil \frac{L}{\delta_x} \rceil$. In addition, we define the cell interfaces $\{x_{j+1/2}\}_{j \in \{0, \dots, N_x\}}$ by $x_{j+1/2} := j\delta_x$, $j \in \{0, \dots, N_x\}$ and the time mesh $t^n = n\delta_t$, $n \in \{0, \dots, N_t\}$, $N_t := \lceil \frac{T}{\delta_t} \rceil$. We discretize the initial datum $\rho_0 \in L^\infty(\mathbb{R})$ on the domain $[0, L]$ by the mean value of the initial datum in a cell as

$$\rho_i^0 = \frac{1}{\delta_x} \int_{x_{i-1/2}}^{x_{i+1/2}} \rho_0(y) dy, \quad i \in \{1, \dots, N_x\}$$

and assume that the approximate solution is of the following dimension

$$\rho \in \mathbb{R}^{N_x \times (N_t+1)}.$$

If we then integrate Eq. (6) over space and time inside the domain $[0, L]$ we obtain the exact relation

$$\rho_i^{n+1} = \rho_i^n - \frac{\delta t}{\delta x} \left(F_{i+1/2}^n - F_{i-1/2}^n \right), \quad n \in \{0, \dots, N_t - 1\}, \quad i \in \{1, \dots, N_x\}, \quad (7)$$

where $F_{i+1/2}^n$ is defined as

$$F_{i+1/2}^n = \frac{1}{\delta t} \int_{t_n}^{t_{n+1}} f(\rho(x_{i+1/2}, t)) dt, \quad i \in \{1, \dots, N_x\} \quad (8)$$

for f as in Eq. (6). In other words, $F_{i+1/2}^n$ is the flux that passes from cell i to $i+1$ between t_n and t_{n+1} . Eq. (7) is a weaker version of Eq. (1) as information is lost through integration, we get the whole information back as the grid is refined. Note that the use of Eq. (7) in the learning process is a key element of this work since it provides a physical structure inside the learned neural network model. Part of the criticism of neural network approaches is that some of the physical structure of the underlying physical phenomena is lost in the structure of the neural network. Here, mass conservation is ensured by the explicit use of this structure in the learning procedure. It not only provides some “explainable AI” outcomes (we can recover the flux function later as illustrated in Section 5.2.1), but also helps with the accuracy of the numerical results.

The behavior at the boundary is important, however, we focus in this work only on what happens inside the specified domain $[0, L]$ and its corresponding grid. This is why we will project values outside of the defined domain onto the boundary values in our numerical algorithm. To this end, we define the following projection operator

$$p : \begin{cases} \mathbb{Z} \rightarrow \{1, \dots, N_x\} \\ i \mapsto \begin{cases} i & \text{if } i \in \{1, \dots, N_x\} \\ 1, & \text{if } i \in \mathbb{Z}_{\leq 0} \\ N_x, & \text{if } i \in \mathbb{N}_{\geq N_x}. \end{cases} \end{cases} \quad (9)$$

As we aim for numerically approximating with a finite volume scheme, we define additionally a general first-order finite volume scheme for $(a, b) \in \mathbb{N}_{\geq 0} \times \mathbb{N}_{\geq 1}$ the chosen stencil size and a function $F \in W_{\text{loc}}^{1,\infty}(\mathbb{R}^{a+b+1}; \mathbb{R})$, by its recursive acting on ρ as follows for $(n, i) \in \{0, \dots, N_t\} \times \{1, \dots, N_x\}$

$$\rho_i^{n+1} = \rho_i^n + \frac{\delta_t}{\delta_x} \left(F(\rho_{p(i-a)}^n, \dots, \rho_{p(i)}^n, \dots, \rho_{p(i+b)}^n) - F(\rho_{p(i-a-1)}^n, \dots, \rho_{p(i)}^n, \dots, \rho_{p(i+b-1)}^n) \right). \quad (10)$$

Then, the solution ρ of Eq. (6) can be numerically approximated by

$$\rho(t, x) \approx \sum_{n=1}^{N_t} \sum_{i=1}^{N_x} \chi_{[x_{i-1/2}, x_{i+1/2})}(x) \chi_{[t_{n-1}, t_n)}(t) \rho_i^n, \quad (t, x) \in (0, T) \times \mathbb{R}.$$

Remark 2 (*Structure and conservation of mass: Physically inspired*). The scheme in Eq. (10), as common for finite volume schemes, conserves “mass” by definition, so that when later optimizing the function F (see Section 3.4 and Section 3.3), we have this property “hard” coded. Thus, we can interpret this approach as *physically inspired*.

Remark 3 (*The CFL-Condition: Physically inspired*). The choice of $\delta_t \in \mathbb{R}_{>0}$ and $\delta_x \in \mathbb{R}_{>0}$ is of course not arbitrary but needs to satisfy the CFL condition (compare again [9]) which boils down to that the time grid needs to be fine enough (in comparison to the space grid) so that

$$\frac{\delta_t}{\delta_x} \leq \frac{1}{\|f'\|_{L^\infty}}$$

is satisfied for numerical stability. The L^∞ -norm of the right-hand side needs to be taken only over the values actually attained which however might not be known a priori. Neither f nor the actually attained values might be known so that a rather small time stepping might be required a priori. On the other hand, the time update in Eq. (10) is designed in a way that is to be expected from a hyperbolic system as such under consideration, so once more, we partially give a structure to the scheme to be “learned”, so that the wording *physically inspired* is reasonable here as well.

3.2. Classical numerical methods

In this subsection, we will shortly introduce classical finite volume schemes for approximating solutions of Eq. (6).

3.2.1. Lax-Friedrichs scheme

The Lax-Friedrichs scheme (see [9, Chapter 4.6, The Lax-Friedrichs method]) provides stability by adding a *numerical diffusion* term to the flow that vanishes when the grid is refined. After some rearrangement of terms, one obtains its conservative form as

$$F_{\text{LxF}} : (\rho_1, \rho_2) \mapsto \frac{1}{2} (f(\rho_1) + f(\rho_2)) - \frac{\delta_x}{\delta_t} (\rho_2 - \rho_1)$$

and can then chose for $(a, b) = (0, 1)$ and F in Eq. (10) as

$$F(y, z) = F_{\text{LxF}}(y, z), \quad \forall (y, z) \in \mathbb{R}^2.$$

This method however introduces diffusion to the solutions that makes the approximation less precise unless the grid is very fine.

3.2.2. Godunov scheme

Godunov’s method [54] approximates at each cell interface the Riemann problem.

In the case of a concave flux f , it reaches its only maximum at the critical density ρ_c as defined in Eq. (5) and the numerical flux F_G can be written explicitly as

$$F_G(\rho_1, \rho_2) = \begin{cases} f(\rho_2) & \text{if } \rho_c < \rho_2 < \rho_1 \\ f(\rho_c) & \text{if } \rho_2 < \rho_c < \rho_1 \\ f(\rho_1) & \text{if } \rho_2 < \rho_1 < \rho_c \\ \min(f(\rho_1), f(\rho_2)) & \text{if } \rho_1 \leq \rho_2. \end{cases}$$

Thus, when choosing $(a, b) = (0, 1)$, the numerical flux function $\mathcal{F}$ in Eq. (10) can be written as follows

$$\mathcal{F}(y, z) = \mathcal{F}_G(y, z), \quad \forall (y, z) \in \mathbb{R}^2.$$

3.2.3. Engquist-Osher scheme

The Engquist-Osher scheme, developed in [55], is also based on solving the Riemann problem at each interface, it is as such close to Godunov's scheme. The difference lies in the “transonic” shock case where $\rho_i^n \leq \rho_c < \rho_{i+1}^n$ (see [9, Chapter 12.8] and Eq. (5) for the critical density ρ_c).

For a concave flux f , its numerical flux function $\mathcal{F}_{EO}$ can be explicitly written as

$$\mathcal{F}_{EO}(\rho_1, \rho_2) = \begin{cases} f(\rho_1) & \text{if } \rho_2, \rho_1 \leq \rho_c \\ f(\rho_2) & \text{if } \rho_2, \rho_1 > \rho_c \\ f(\rho_c) & \text{if } \rho_2 \leq \rho_c < \rho_1 \\ f(\rho_1) + f(\rho_2) - f(\rho_c) & \text{if } \rho_1 \leq \rho_c < \rho_2. \end{cases}$$

Then, choosing once more $(a, b) = (0, 1)$ the numerical flux function $\mathcal{F}$ in Eq. (10) can be chosen as

$$\mathcal{F}(y, z) = \mathcal{F}_{EO}(y, z), \quad \forall (y, z) \in \mathbb{R}^2.$$

3.3. Learning numerical schemes with neural networks

In this section, we develop one of the key contributions of this article: a general framework using *Neural Networks* (NNs) to learn first-order numerical schemes that would produce accurate approximations of solutions to the conservation law (1). We begin in Section 3.3.1 and Section 3.3.2 by defining a general NN architecture that can in principle approximate any continuous (a, b) -stencil function on a compact set (see the $\mathcal{F}$ in Eq. (10)). We present in Section 3.3.3 three applications of this architecture to learning new numerical schemes for conservation laws. We will in the following parts mostly use this framework to approximate $(0, 1)$ -stencil numerical flux functions as described in Section 3.1, but the strength of this framework is precisely that one can easily extend the stencil to get better approximation or to apply this framework to other problems.

3.3.1. Approximating (a, b) -stencil functions with dense neural networks

We begin by defining here a dense neural network denoted $\mathcal{AB}_p$, characterized by its parameters p . This neural network defines an (a, b) -stencil scalar function computing a given scalar from the local information contained in the (a, b) -stencil $(\rho_{i-a}, \dots, \rho_i, \dots, \rho_{i+b})$, $i \in \{1, \dots, N_x\}$ from the discretized state $\rho \in \mathbb{R}^{N_x}$.

Definition 3.1. A dense neural network $\mathcal{AB}_p$ of depth $d \in \mathbb{N}_{\geq 1}$ with hidden dimension $h \in \mathbb{N}_{\geq 1}$ is defined as the non-linear function

$$\mathcal{AB}_p : \mathbb{R}^{a+b+1} \rightarrow \mathbb{R}$$

$$(\rho_{i-a}, \dots, \rho_i, \dots, \rho_{i+b}) = X \mapsto \sigma_d(W_d \cdot + B_d) \circ \dots \circ \sigma_j(W_j \cdot + B_j) \circ \dots \circ \sigma_0(W_0 X + B_0)$$

with

$$\begin{cases} W_0 \in \mathbb{R}^{h \times (a+b+1)}, B_0 \in \mathbb{R}^h \\ W_j \in \mathbb{R}^{h \times h}, B_j \in \mathbb{R}^h \\ W_d \in \mathbb{R}^{1 \times h}, B_d \in \mathbb{R}, \end{cases} \quad \forall 1 \leq j \leq d-1$$

where $p = (W_0, B_0, \dots, W_d, B_d)$ are the network parameters: respectively named weights and biases, and $\sigma_j : \mathbb{R} \rightarrow \mathbb{R}$ the *scalar* activation functions applied component-wise.

An example representation of this dense neural network can be found in Fig. 1.

We can then optimize the parameters p in order to get $\mathcal{AB}_p$ to approximate any given (a, b) -stencil scalar function. This is possible using modern deep learning frameworks [56–58] that implement highly efficient and scalable gradient computing and optimization algorithms.

3.3.2. A convolutional architecture to efficiently compute predictions

In the present work, we focus on a local scale where the numerical schemes are applied. From a theoretical point of view, we will only use an (a, b) -stencil dense NN (3.1) to approximate the numerical flux in first-order schemes. To compute the prediction of our neural network $\mathcal{AB}_p$ on the entire grid, it should, however, be noted that we use a 1-dimensional CNN architecture as an efficient way to apply our dense neural network function $\mathcal{AB}_p$ on all the stencils from the input state $\rho^n \in \mathbb{R}^{N_x}$.

A *convolutional neural network* (CNN) [59] is a neural network architecture whose layers convolve or “slide” a lower-dimensional kernel across input data of potentially higher dimensions. Consequently, this architecture yields predictions that are both locally dependent and invariant with respect to their position within the input. This “sliding” operation is very similar to applying a local

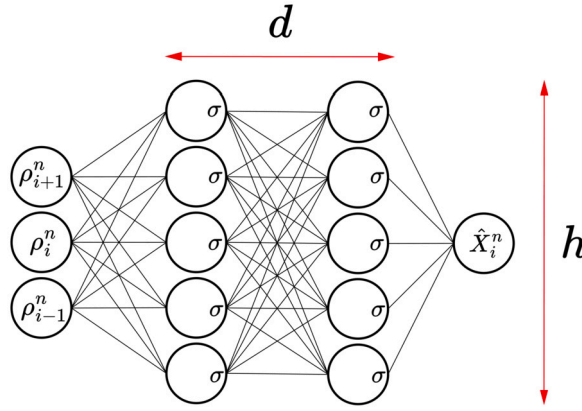


Fig. 1. (1,1)-stencil dense neural network with $d = 2$, $h = 5$ computing a given scalar $\hat{X}_i$ from local information around cell ρ_i .

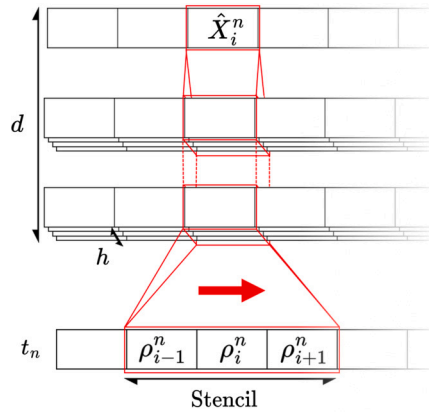


Fig. 2. Illustration of the use of a CNN architecture for applying a dense (1,1)-stencil neural network with depth d and hidden dimension h (see Definition 3.1) over ρ^n at time t_n to compute a given quantity $\hat{X}^n$.

numerical scheme on a state $\rho^n \in \mathbb{R}^{N_x}$ as depicted in Fig. 2. Choosing carefully the right parameters for the convolution layers, we are indeed able to use a CNN architecture to apply our (a, b) -stencil dense NN $\mathcal{AB}_p$ on input states of any size. The use of existing CNNs implementations in this work allowed us to leverage highly optimized code that parallelizes the matrix multiplications for all the stencils in the inputs states¹ as well as the tracking of the computational graph of the predictions that we need in order to later perform backpropagation on the NN parameters.

3.3.3. Developing neural network-based schemes for forward prediction

We can now apply the previously defined neural network model to learning numerical schemes for approximating solutions of the conservation law in Eq. (1) with initial discretized state $\rho^0 \in \mathbb{R}^{N_x}$.

Brute force scheme learning The first naive approach is to model the entire scheme into our (a, b) -stencil NN and compute the state ρ^{n+1} from ρ^n as

$$\rho_i^{n+1} = \mathcal{AB}_p(\rho_{i-a}^n, \dots, \rho_i^n, \dots, \rho_{i+b}^n), \quad \forall (n, i) \in \{0, \dots, N_t\} \times \{1, \dots, N_x\}$$

with the fully-connected (a, b) -stencil neural network $\mathcal{AB}_p$ introduced in Definition 3.1. Note that we are able to write this for all $i \in \{1, \dots, N_x\}$ including the boundaries leveraging the projection operator defined in Section 3.1 that gives a value to use for the cells outside our working domain.

However, this approach ignores the structure of the conservation laws in Eq. (1) and especially the conservation of mass whose structure in the framework of finite volume methods is detailed in Eq. (7). With this method, the function space of $\mathcal{AB}_p$ includes a majority of undesired non-conservative schemes. As a result, in the upcoming sections, we will introduce alternative models that enforce conservation of mass by design.

¹ We perform batched prediction, so we have multiple input states to apply our models on.

Flux-learning method One way to ensure that the system is conservative consists of enforcing a scheme in conservation form (see for that Eq. (7)) and modeling only the numerical flux as a neural network. The scheme now reads

$$\rho_i^{n+1} = \rho_i^n - \frac{\delta_t}{\delta_x} (\hat{F}_i^n - \hat{F}_{i-1}^n), \quad \forall (n, i) \in \{0, \dots, N_t\} \times \{1, \dots, N_x\}, \quad (11)$$

where $\hat{F}_i^n$ is an approximation of the numerical flux $F_{i+1/2}^n$ at $x_{i+1/2}$ i.e. between cells i and $i+1$. It can be computed with any given stencils. Using the (a, b) -stencil NN defined in Definition 3.1 we can define $\hat{F}_i^n \in \mathbb{R}$ as

$$\hat{F}_i^n = \mathcal{AB}_p(\rho_{i-a}^n, \dots, \rho_i^n, \dots, \rho_{i+b}^n) \approx F_{i+1/2}^n, \quad \forall (n, i) \in \{0, \dots, N_t\} \times \{1, \dots, N_x\}. \quad (12)$$

This method works well in practice as will be demonstrated in the following sections. Even so, the underlying optimization problem has no unique solution. In fact, if an optimal numerical flux exists, then it is not unique as long as shifting any flux function by a constant would yield exactly the same results. One can clearly see that this constant would cancel out in Eq. (11). This is not an issue for the present work if we only want to perform prediction, but can become one if we are interested in extracting real physical values from the learned flux. This is further discussed in Section 5.2.1.

Speed-learning method In the continuous case, the continuity equation links the speed v , the flow f and the density functions ρ as

$$f(t, x) = \rho(t, x)v(t, x), \quad \forall (t, x) \in [0, T] \times [0, L].$$

In the *discrete case* using finite volume methods, the continuity equation still holds if we define v_i^n as the space-averaged velocity in the cell i at time t_n , such that

$$F_{i+1/2}^n = \rho_i^n v_i^n.$$

A numerical proof can be found in [60]. Then, we can learn the aggregated speeds on all cells $v^n \in \mathbb{R}^{N_x}$ rather than the fluxes: and then in turn compute the fluxes with the continuity equation. This novel approach introduces additional physics-inspired structure into the neural network model, and notably enforces a numerical flux of 0 in cells without density, addressing the floating constant concern present in the flow model outlined in Eq. (11). The speed-learning method therefore implements the scheme

$$\rho_i^{n+1} = \rho_i^n - \frac{\delta_t}{\delta_x} (\rho_i^n \hat{v}_i^n - \rho_{i-1}^n \hat{v}_{i-1}^n), \quad \forall (n, i) \in \{0, \dots, N_t\} \times \{1, \dots, N_x\}, \quad (13)$$

where the aggregated speed on cell i , v_i^n , is estimated by $\hat{v}_i^n$ with a model. Using for instance the (a, b) -stencil NN (3.1) would result in computing

$$\hat{v}_i^n = \mathcal{AB}_p(\rho_{i-a}^n, \dots, \rho_i^n, \dots, \rho_{i+b}^n) \approx v_i^n \in \mathbb{R}, \quad \forall (n, i) \in \{0, \dots, N_t\} \times \{1, \dots, N_x\}. \quad (14)$$

3.4. Polynomial approximations for numerical flux functions

Due to the expressivity of neural networks with many parameters and in order to get an insight into the complexity of the learning problem, it can be insightful to compare them to other function approximators. To this end, we introduce a particularly simple class of polynomials.

The following setup will serve as a baseline for comparing the more advanced function space of neural networks with a rather primitive approach.

For a maximum density $\rho_{\max} \in \mathbb{R}_{>0}$, we consider a numerical flux function

$$F_p : [0, \rho_{\max}]^2 \rightarrow \mathbb{R},$$

where F_p has a representation

$$F_p(\rho_{-1}, \rho) = g_p(\rho_{-1})h_p(\rho), \quad \forall \rho_{-1}, \rho \in [0, \rho_{\max}] \quad (15)$$

and $g_p, h_p : [0, \rho_{\max}] \rightarrow \mathbb{R}$ are differentiable functions that depend on some parameter $p \in \mathbb{R}^m$, where $m \in \mathbb{N}$. Such functions are often called rank-1 functions and are a particularly simple function class that lets us pick well-conditioned parametrizations and include constraints.

In our experiments, we pick $m \in 2\mathbb{N}$, and $p \in \mathbb{R}^m$ such that $(x_1, p_1), \dots, (x_{m/2}, p_{m/2}) \in [0, \rho_{\max}] \times \mathbb{R}$ are interpolation points for a monotonic piecewise cubic Hermite interpolating polynomial g_p using the method in [61]. We construct h_p similarly with the interpolation points $(x_{m/2+1}, p_{m/2+1}), \dots, (x_m, p_m) \in [0, \rho_{\max}] \times \mathbb{R}$.

4. Training data and objective functions

4.1. Obtaining the numerical flux of a discretized solution

To motivate our goal of finding a function that restricts a numerical flux, which generates good approximations of discretized solutions, we discuss the numerical flux field that can be recovered from a solution to Eq. (1). Understanding the limitations in the use of this recovered numerical flux field is important in deriving the approach of this work.

Consider a discretization of such a solution $\rho \in \mathbb{R}^{N_x \times N_t + 1}$ as introduced in Section 3.1. We define for all $n \in \mathbb{N}_{\leq N_t}$, the numerical flux field $F_{\cdot-1/2}^n \in \mathbb{R}^{N_x+1}$ implicitly by

$$\begin{aligned} F_{i-1/2}^n - F_{i+1/2}^n &:= \frac{\delta_x}{\delta_t} (\rho_i^{n+1} - \rho_i^n), \quad \forall i \in \{1, \dots, N_x\} \\ \Leftrightarrow \begin{pmatrix} 1 & -1 & & \\ & \ddots & \ddots & \\ & & 1 & -1 \end{pmatrix} F_{\cdot-1/2}^n &= \frac{\delta_x}{\delta_t} (\rho^{n+1} - \rho^n). \end{aligned}$$

Due to the matrix being of rank N_x , the numerical flux field is well-defined once we specify a numerical boundary flux. If solutions have constant boundary density, we determine the numerical boundary flux by specifying the flux f in the boundary, i.e.,

$$F_{N_x+1/2}^n = f(\rho_{N_x}^n, \rho_{N_x}^n).$$

Consequently, we obtain an exact numerical flux field, we may call the *observed numerical flux*, by obtaining the unique solution of the system

$$\begin{aligned} F_{i-1/2}^n &= \frac{\delta_x}{\delta_t} (\rho_i^{n+1} - \rho_i^n) + F_{i+1/2}^n, \quad \forall i \in \{1, \dots, N_x - 1\} \\ F_{N_x+1/2}^n &= \frac{\delta_x}{\delta_t} (\rho_{N_x}^n - \rho_{N_x}^n) + f(\rho_{N_x}^n, \rho_{N_x}^n). \end{aligned}$$

It should be noted that in general there does not exist an $\mathcal{F} : \mathbb{R}^2 \rightarrow \mathbb{R}$ such that the observed numerical flux field is a function of only the stencil, i.e.,

$$F_{i-1/2}^n \neq \mathcal{F}(\rho_{i-1}^n, \rho_i^n), \quad \forall i \in \{1, \dots, N_x\} \quad \forall n \in \mathbb{N}_{\leq N_t}.$$

This means that determining a numerical flux function that generates accurate discretized solutions to Eq. (1), is inherently an approximation problem.

For piecewise constant initial data Godunov's method using an exact Riemann solver is exact for two grid cells in time, i.e., $N_t = 1$, given the CFL condition is fulfilled. In this setup, all values of $\mathcal{F}$ are exactly determined by a family of Riemann problems.

Therefore, it is easily observed by the inexactness of Godunov's method for time horizons $N_t > 1$ that there is no such unique numerical flux function that is exact, while not depending on t , x , or further variables. By this argument, we conclude that approximating the numerical flux function directly or approximating single time steps of the discretized system is an ill-posed objective toward obtaining a numerical flux function that produces approximate solutions. In fact, the topology in which we would approximate a recovered numerical flux is unknown in this problem formulation.

4.2. Full time horizon approximation

Given that as our global objective is to perform accurate predictions over arbitrarily large time horizons: it is reasonable to formulate an optimization framework that considers a sequence of chained predictions from the learning model. While it is feasible to enhance performance under a fixed discretization, such improvements might lack uniformity across discretization levels. For instance, optimizing the prediction on only the subsequent state may result in numerical methods that diverge after a limited number of timesteps. This is why we trained our models over several timesteps, in a recurrent fashion. The model's errors can thus propagate and amplify through the successive predictions, allowing the optimization procedure to focus on learning features that are consistent over time. This leads to an optimization algorithm for our NN models that closely resemble *backpropagation through time* [62] described in Section 4.2.3.

4.2.1. Training data

We aim to optimize schemes to compute accurate solutions for first-order conservation laws. In the present work, we apply our methods to optimize first-order numerical schemes for the Lighthill-Whitham-Richards (LWR) model [63,64] with a given concave flux f .

We firstly focus our attention on the Greenshield flux, described in Section 3.1, one of the most commonly used in traffic theory. It features rarefaction waves that can be difficult to solve with first-order numerical schemes. In short, we aim at approximation solutions to the conservation law

$$\partial_t \rho(t, x) + \partial_x \left(v_{\max} \rho(t, x) \left(1 - \frac{\rho(t, x)}{\rho_{\max}} \right) \right) = 0 \quad (16)$$

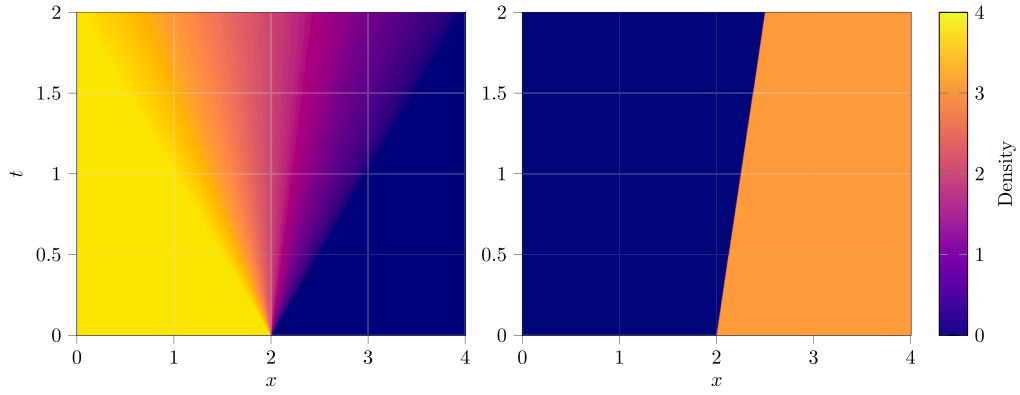


Fig. 3. Two Riemann solutions from the training set. Left: A rarefaction wave. Right: A shock wave, discretized on 800×400 grids: $\delta x = \delta t = 0.005$.

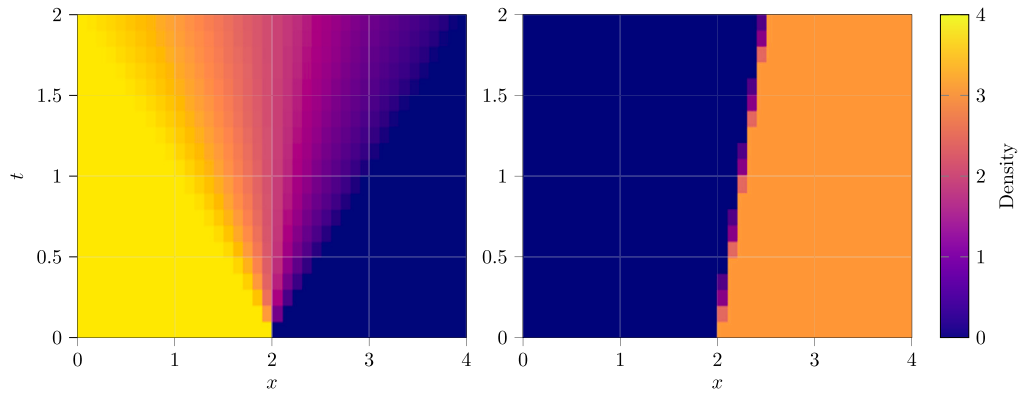


Fig. 4. Same solutions coarsened to 40×20 grids: $\delta x = \delta t = 0.1$.

with $v_{\max} = 1 \text{ m.s}^{-1}$ the free-flow speed and $\rho_{\max} = 4 \text{ veh/m}$ the maximum density value. In our work, we will only train our models on fundamental solutions of Eq. (16) and show that the model generalizes to computing solutions to more general initial data. We generate a training set using the known closed forms of the solutions for Riemann problems (Section 2.1), constituting what we claim to be a representative set of *shock* and *rarefaction waves*, discretized according to the finite volume method framework described in Section 3.1. One can find two example solutions depicted both for a high-resolution grid in Fig. 3, and for a lower resolution in Fig. 4.

To build our training data set, we sampled $380 = 20 \cdot (20 - 1)$ evenly spaced points

$$\{(u_l, u_r) \in [0, \rho_{\max}]^2 : u_l \neq u_r\}$$

and then generated the associated solutions discretized on a grid using the known closed-form expressions of the solutions defined in Section 2.1 which we can evaluate on any grid of any size. 380 is a trade-off between a representative dataset and a reasonable size that allows optimization within appropriate computation time. All grids have the same parameters, i.e., $\delta x = \delta t = 0.01$, $N_x = 802$ cells, and $N_t = 400$ timesteps.

The crucial choice here is N_t i.e. the chosen number of timesteps in the training data. As explained in Section 4.2, having a very long time horizon leads *a priori* to better results as the errors of the scheme can propagate further. Here, we then choose a value for N_t as high as possible, while allowing optimization on such a time horizon. We put the initial discontinuity in the middle of the domain $[0, \delta_x N_x]$, our models being translationally invariant. One can note that we use here $N_x = 2N_t$ because we are dealing with maximum wave speed $v_{\max} = 1$ and want to ensure that the solutions do not contain waves that hit the border i.e., that we do not have to care about the boundaries during training: we are only interested in what happens inside the domain i.e., the propagation of the initial discontinuity. Note that after what was said before, the values of $\delta x, \delta t$ do not affect the content of the training data, only their ratio does.

Fig. 5 illustrates the frequency and position of numerical flux samples, i.e., stencils used in the optimization of the procedure. The initial data are evenly sampled Riemann problems, however, stencils will not be evenly distributed—even for the considered initial data. Over time the physical effects of the law at hand inherently distribute stencils unevenly, e.g., by smoothing rarefaction waves but not shocks. An inherently uneven data distribution is an important characteristic of physical laws.

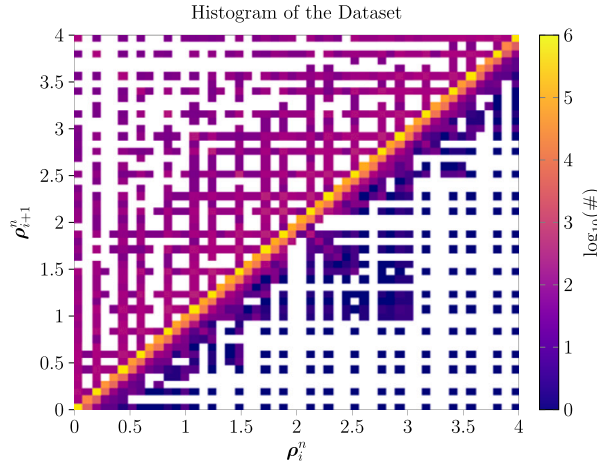


Fig. 5. Histogram of spatially successive density values (binned in a 50×50 grid) in the dataset used for computing approximate solvers. The color represents the base-10 logarithm of the count.

4.2.2. Objectives

We now define the objective function that we will minimize with optimization. For a given discretization grid, it is common in machine learning to use as an objective function the *mean square error* (MSE), or mean squared ℓ^2 -norm, between the prediction of the scheme on any of the $K \in \mathbb{N}$ initial data from the training set $\{(\rho^0)^k \in \mathbb{R}^{N_x} : k \in \{1, \dots, K\}\}$ (discretized step functions), and the *exact discretized solutions on the subsequent timesteps* $\{(\rho)^k \in \mathbb{R}^{N_x \times (N_t+1)} : k \in \{1, \dots, K\}\}$, where we use $(\rho)^k$ to denote the k -th element in our training set.

For any estimation $\hat{A} \in \mathbb{R}^D$, $D \in \mathbb{N}_{\geq 1}$ and corresponding ground truth value $A \in \mathbb{R}^D$ the *mean square error* (MSE) between $\hat{A}$ and A can be expressed as

$$\text{MSE}(\hat{A}, A) = \frac{1}{D} \|\hat{A} - A\|_2^2 = \frac{1}{D} \sum_{i=1}^D (\hat{A}_i - A_i)^2. \quad (17)$$

However, in our case, an error at timestep $n = 1$ is less critical than an error at timestep $n = N_t$. The main reason is that we are interested in predicting solutions on large time horizons. Also, one can argue that a model can predict the solution at time N_t if it can predict a good solution for lower timesteps. Inspired by the “discount factor” used in reinforcement learning techniques [65], we then choose to use a factor that emphasizes errors at higher timesteps.

Denoting $\{(\hat{\rho})^k \in \mathbb{R}^{N_x \times (N_t+1)} : k \in \{1, \dots, K\}\}$ the approximated solutions computed using our scheme, our training loss can finally be written as

$$\text{Loss}((\hat{\rho})_{k \in [1, K]}^k, (\rho)_{k \in [1, K]}^k) = \sum_{n=0}^{N_t} \frac{\gamma^{N_t}}{K} \gamma^{-n} \sum_{k=1}^K \text{MSE}((\hat{\rho}^n)^k, (\rho^n)^k) \quad (18)$$

$$= \sum_{n=0}^{N_t} \frac{\gamma^{N_t}}{K N_x} \gamma^{-n} \sum_{k=1}^K \left\| (\hat{\rho}^n)^k - (\rho^n)^k \right\|_2^2 \quad (19)$$

with a typical decay factor $\gamma = 0.99$ that still give reasonable weight to the first predictions ($0.99^{100} \approx 0.37$).

4.2.3. Optimization problem and algorithm

Altogether, we aim for solving the following nonlinear constrained optimization problem for $\delta_t, \delta_x \in \mathbb{R}_{>0}$ given.

We will suppress the projection operator in the finite volume update Eq. (10) and only consider a $(0, 1)$ -stencil to keep notation simple:

$$\min_{\hat{\rho}^k, p} \text{Loss}((\hat{\rho})_{k \in [1, K]}^k, (\rho)_{k \in [1, K]}^k) = \sum_{n=0}^{N_t} \frac{\gamma^{N_t}}{K N_x} \gamma^{-n} \sum_{k=1}^K \left\| (\hat{\rho}^n)^k - (\rho^n)^k \right\|_2^2 \quad (20)$$

subject to (for all $k \in \{1, \dots, K\}$)

$$\begin{aligned} (\hat{\rho}_i^0)^k &= (\rho_i^0)^k, & i &\in \{1, \dots, N_x\} \\ (\hat{\rho}_i^{n+1})^k &= (\hat{\rho}_i^n)^k + \frac{\delta_t}{\delta_x} \left(\mathcal{F}_p((\hat{\rho}_i^n)^k, (\hat{\rho}_{i+1}^n)^k) - \mathcal{F}_p((\hat{\rho}_{i-1}^n)^k, (\hat{\rho}_i^n)^k) \right), & (i, n) &\in \{1, \dots, N_x\} \times \{1, \dots, N_t\} \end{aligned} \quad (21)$$

where $\mathcal{F}_p$ is defined by our previously introduced approximations, $\forall (x, y) \in \mathbb{R}^2$

$$F_p(x, y) \equiv \begin{cases} \mathcal{AB}_p(x, y) & \text{for the (0, 1)-Flow model Eq. (12),} \\ x \cdot \mathcal{AB}_p(x, y) & \text{for the (0, 1)-Speed model Eq. (13),} \\ g_p \cdot h_p(x, y) & \text{for the polynomial basis as in Eq. (15).} \end{cases} \quad (22)$$

Each sample of the training data is a sequence of discretized states of the solution. In order to compute a prediction on the entire time horizon, any numerical scheme predicts new states one by one, taking as input for time t_n the previously-predicted state at time t_{n-1} (see Eq. (22)). This makes the optimization problem very hard as long as the same parameters are used for each prediction, and a slight modification at time t can lead to huge changes and badly conditioned non-linear dependencies after a small number of timesteps. Computing the gradients of the loss w.r.t. the parameters over a given time horizon is a well-known topic in the deep learning community. For NN, the corresponding algorithm is usually called *backpropagation through time* and was first explored by Werbos in 1988 [62]. This technique is still hard to implement because of well-known *vanishing* or *exploding gradients* issues when backpropagating gradients through time. We provide a pseudo-code describing the algorithm used to train the neural networks in Algorithm 1 (see also [66] for the optimizer used). It is very similar to *backpropagation through time* while implementing additional features such as time decay.

Algorithm 1 Decayed Backpropagation through time.

```

Require:  $N_e \geq 0$  ▷ The number of epochs
Require:  $K \geq 0$  ▷ The training dataset size
Require:  $N_t \geq 0$  ▷ The number of timesteps in the data
Require:  $\gamma \leq 1$  ▷ The decay factor
for  $e = 1$  to  $N_e$  do
  Set total loss L to 0
  for  $k = 1$  to  $K$  do
     $\rho \leftarrow (\rho)^k$  ▷ get the  $k$ -th training solution
     $\hat{\rho}^0 \leftarrow \rho^0$  the initial state
     $L_k \leftarrow 0$  ▷ Set the partial loss on the  $k$ -th training sample to 0
    for  $n = 1$  to  $N_t$  do
      Compute the state  $\hat{\rho}^n$  from  $\hat{\rho}^{n-1}$  with model
       $L_k \leftarrow \gamma L_k + \text{MSE}(\hat{\rho}^n, \rho^n)$ 
    end for
     $L \leftarrow L + L_k / K$ 
  end for
  Compute gradient of  $L$  w.r.t. model parameters
  Update Model parameters with ADAM optimizer [66]
end for

```

4.2.4. Training routine

To practically perform training, we still have to fix some parameters. Firstly, the optimization problem variables, i.e., how many Riemann waves to generate and on what time-horizon to train. There are also the practical model training parameters, i.e., how many *epochs* N_e ,² what *learning rate* or *step size* the optimizer should use, etc. In practice, the longer the time horizon is, the slower the training will be. Propagating gradients through a large time horizon is memory intensive, and also slow. There is therefore a trade-off between training speed/efficiency on small time-horizon and better approximation with longer time horizons. We ended up training on an intermediate number of 100 timesteps for a first fast and coarse training, and then slowly fine-tuning the models on higher precision and larger time horizons, up to 500 timesteps.

Our training set is made of an equal amount of shock and rarefaction waves. Rarefactions are however much more complex to handle as they spread over space whereas shocks, albeit moving, stay localized. With that in mind, we choose to train our models to fit only rarefaction waves after the initial fit, and before fine-tuning the models on the whole training set. This focus on rarefactions has proven to be an efficient way to “guide” the model to better optima through a loss landscape that contains potentially many local minima, typical for recurrent neural networks.

4.2.5. Architecture of neural network models

We now apply our neural network architectures, previously defined in Section 3.3.3, to this framework. We aim at benchmarking our models against classical (1, 1)-stencil schemes that use (0, 1)-stencil flux functions (namely the Lax-Friedrichs scheme defined in Section 3.2.1, Godunov scheme in Section 3.2.2 and Engquist-Osher scheme in Section 3.2.3). In order to have a fair comparison with those first-order schemes, we choose also to model (0, 1)-stencil flux functions. Also, as Eq. (16) defines a local flux $f(\rho(x))$, the numerical flux $F_{i+1/2}^n$ should not depend on values too far away from the cell interface at $x_{i+1/2}$.

For neural networks models defined in Section 3.3.3, we also have to choose the network size and the activation functions that we use for the $\mathcal{AB}_p$ NN defined in Definition 3.1. Those choices are the fruit of theoretical and physical insights as well as practical assessments of the model’s capabilities.

² In deep learning, an epoch refers to one complete cycle through the entire training dataset during the training of a model.

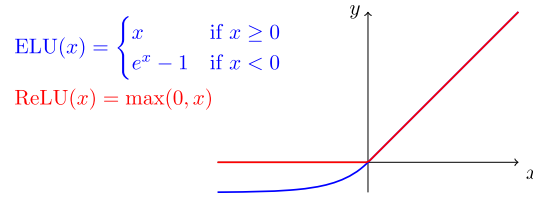


Fig. 6. Comparison between ELU and ReLU activation functions.

Network parameters: In order to ensure sufficient approximation capabilities and reduce the chance of overfitting our training set, we chose to use a NN architecture with a reasonable number of parameters allowing us to model a range of simple functions. After some trials, we finally chose to use a depth $d = 6$ and a hidden dimension $h = 15$ which corresponds to a total of 1261 parameters.³

Activation functions: Backpropagation through time [62] involves computing the gradients of the loss w.r.t. the model parameters for each successive prediction of the model over time. This means that the algorithm will “backpropagate” the gradients through the network dozens of times during a training step. Bounded activation functions like *Sigmoid* or *Tanh* are known to shrink the gradients at each pass, leading to *vanishing gradients* after a few backpropagation steps through the network (see the pioneering work of [67] for more details). This is why it is usually advised to use unbounded activation functions in this framework, the ReLU activation function is the most used for its simplicity and efficiency. However, the ELU activation function proved to be much more adequate in our case, mainly due to the fact that it is C^1 whereas ReLU is only C^0 . We provide a comparison of the two functions Fig. 6. We then set the *scalar* activation functions $\sigma_j, j \in \{0, \dots, d\}$ defined in Definition 3.1 as

$$\begin{cases} \sigma_j : x \mapsto \text{ELU}(x) & \text{if } 0 \leq j < d \\ \sigma_d : x \mapsto \text{ReLU}(x). \end{cases}$$

We maintain a ReLU activation for the final layer to ensure non-negative output values, which better align with physical considerations. In traffic models, the numerical flux $F_{i+1/2}^n$ is inherently positive due to the exclusion of negative speeds in the modeling process.

With the definition of our dense neural network $\mathcal{AB}_p$ in place, we can readily incorporate it into the corresponding neural network models outlined in Section 3.3.3 in order to learn new numerical schemes.

We will present in the following sections the results and the performance of two neural network models built with this architecture:

- A (0,1)-Flow Model (see Eq. (11)), and
- A (0,1)-Speed Model (see Eq. (13)).

Both of them have the same underlying $\mathcal{AB}_p$ NN architecture with 1261 parameters. They were trained on the discretized Riemann waves set described in Section 4.2.1 with Algorithm 1. The neural networks have been implemented using the software PyTorch [56] and the results can be reproduced using the code available in the provided GitHub repository.⁴

5. Evaluation and comparison with existing classical schemes

5.1. Evaluation methodology

We compare the different numerical schemes on a set of K exact solutions corresponding to initial data $\{(\rho)^k \in \mathbb{R}^{N_x} : k \in \{1, \dots, K\}\}$. The *exact discretized solutions* $\{(\rho)^k \in \mathbb{R}^{N_x \times (N_t+1)} : k \in \{1, \dots, K\}\}$ are defined as in Section 3.1. More precisely, denoting $\{(\hat{\rho})^k \in \mathbb{R}^{N_x \times (N_t+1)} : k \in \{1, \dots, K\}\}$ the discretized solutions computed with the scheme, the average ℓ^2 -error between the prediction and the ground truth is computed as

$$\begin{aligned} \text{Error} \left((\hat{\rho})_{k \in [1, K]}^k, (\rho)_{k \in [1, K]}^k \right) &= \frac{1}{K} \sum_{k=1}^K \left\| (\hat{\rho})^k - (\rho)^k \right\|_{\ell^2} \\ &= \frac{1}{K} \sum_{k=1}^K \sqrt{\delta_x \delta_t \sum_{n=0}^{N_t} \sum_{i=1}^{N_x} ((\hat{\rho}_i^n)^k - (\rho_i^n)^k)^2}. \end{aligned} \quad (23)$$

We aim to compare the numerical schemes at different levels of discretization, this shows that our models did not overfit to the fixed discretization parameters of the training set. The test set is composed of 18 finely discretized solutions on a 3000×3000 grid with $\delta x = \delta t = 10^{-2}$. Solutions were computed with a Lax-Hopf solver (see Section 2.2). In order to get a representative test set, we

³ $1261 = 2 \times 15 + 15 + 5(15 \times 15 + 15) + 15 \times 1 + 1$ counting the dimensionality of the matrices from Definition 3.1.

⁴ See <https://github.com/VictorMorand/FVchemesOptim/>.

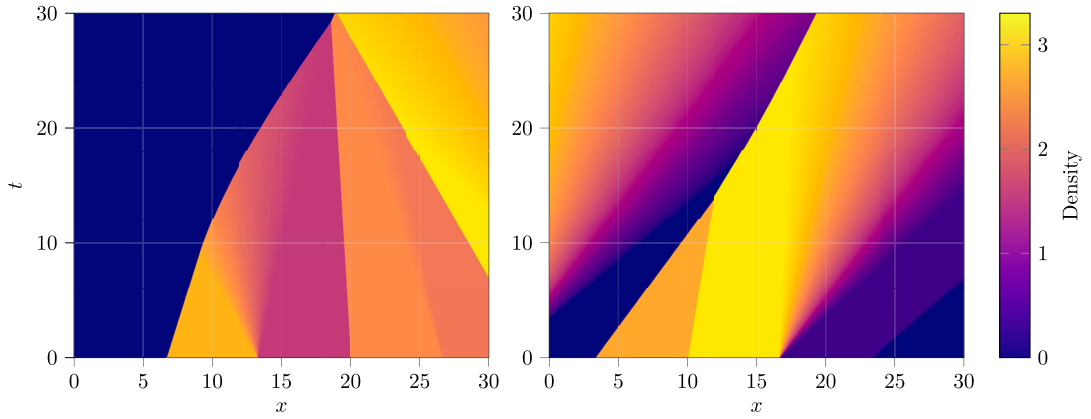


Fig. 7. Two discretized solutions from the test set: $N_t = N_x = 3000, \delta x = \delta t = 10^{-2}$.

generated solutions for random piecewise-constant initial conditions sampled uniformly in $[0, \rho_{\max}]$ detailed in Eq. (24). Two samples are plotted in Fig. 7.

Hard boundary conditions Even if the schemes were designed for initial-value problems, the evaluation is necessarily performed on compact domains. Therefore, we first generated solutions on the domain $(x, t) \in [0, 60] \times [0, 30]$ in Section 3.1, with free-flow boundary conditions, i.e. exact discretized solutions of (16) for the initial condition

$$\forall x \in [1, 60], \rho_0(x) = \sum_{j=0}^9 \mathbb{1}_{[6j, 6(j+1)]}(x) C_j \quad (24)$$

with $C_j \sim \mathcal{U}(0, \rho_{\max})$, $\forall j \in \{0, \dots, 9\}$ where $\mathcal{U}$ stands for the uniform distribution. Then, we restrict the solution to the first region $(x, t) \in [0, 30] \times [0, 30]$, and to the second region $(x, t) \in [30, 60] \times [0, 30]$.

This choice avoids the necessity to generate solutions that fail to attain the value at the boundary, as it may happen for general solutions to initial-boundary value problems, see [68].

Despite this fact, for evaluation purposes, each scheme needs to generate a solution to an initial boundary value problem. Thus, we supplement the initial data with boundary data obtained from the traces of the ground truth solution at the boundary, and enforce during evaluation, the hard boundary condition at each timestep

$$\forall n \in \{1, \dots, N_t\}, \begin{cases} \hat{\rho}_0^n & \leftarrow \rho_0^n \\ \hat{\rho}_{N_x}^n & \leftarrow \rho_{N_x}^n \end{cases} \quad (25)$$

The Lax-Hopf solver produces point-wise discretization and we thus need to aggregate those values to get a proper FVM discretization. If we choose to aggregate $n_x, n_t \in \mathbb{N}_{\geq 1}$ points in space and time respectively into our cells, we get an FVM approximation of the solution with $\delta_x = n_x \times 10^{-2}$ and $\delta_t = n_t \times 10^{-2}$. During evaluation, we then simply aggregate our test set with n_x and n_t progressively increasing, coarsening the discretization of our exact solutions. This restricts us to rational CFLs of the form n_t/n_x , but this is sufficient and enables us to use the same finely discretized explicit solutions which saves a lot of time and power.

5.1.1. Evaluation at fixed CFL

To simplify visualization, we will consider convergence plots with fixed CFL or δ_t/δ_x ratios,⁵ we present in Fig. 8 the convergence plots for two different grid aspect ratios: the standard $\delta_x = \delta_t$ (CFL = 1) on which the models were trained, and another one $\delta_x = 2\delta_t$ (CFL = 1/2). We evaluated our schemes for $n_t \in [3, 150] \cap \mathbb{N}$ which corresponds to a range of time horizons of $N_t \in [20, 1000] \cap \mathbb{N}$, n_x being fixed by the chosen CFL condition.

The optimized first-order schemes outperform the Godunov and Enquist-Osher methods, and not only on the scales and grid parameters they were trained. They also generalize to other discretizations and seem to converge as well with similar convergence rates to the classical first-order schemes. This was achieved without any other knowledge about the PDE than the conservation law. As will be seen in Section 5.2.1, not only did our methods identify an accurate numerical flux function from training data containing solely basic Riemann waves, but also effectively captured the *flux function*, essentially grasping the underlying physics that governed the generation of the training data.

Also, one can note the similarity in performance of the NN speed and flow models. Despite the neural networks attempting to approximate distinct values (speed and flow), we observe remarkably similar schemes. This advocates for the convergence of our schemes to a good optimum for the objective defined in Section 4.2.2. This will be further discussed in Section 5.2.2.

⁵ Here we have a maximum wave velocity of 1, so the CFL is equal to δ_t/δ_x .

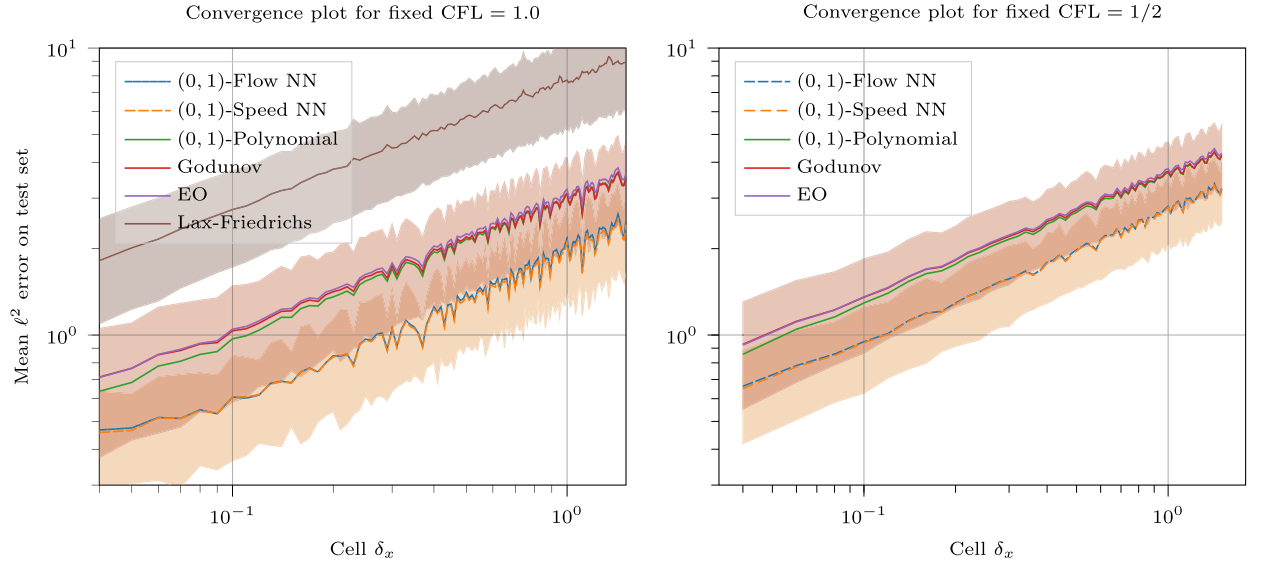


Fig. 8. Evaluation of schemes on LWR-Greenshield Test dataset. Convergence plots showing the mean ℓ^2 error of different schemes on the test set while the grid is refined from 20 to 1000 time-steps. We also present the standard deviation showing the dispersion of the results on the test set (note that we chose to show it only for one model in each group of curves in order to not overload the plot). The (0,1)-Speed and (0,1)-Flow NN models are indistinguishable from one another for $CFL = 1/2$.

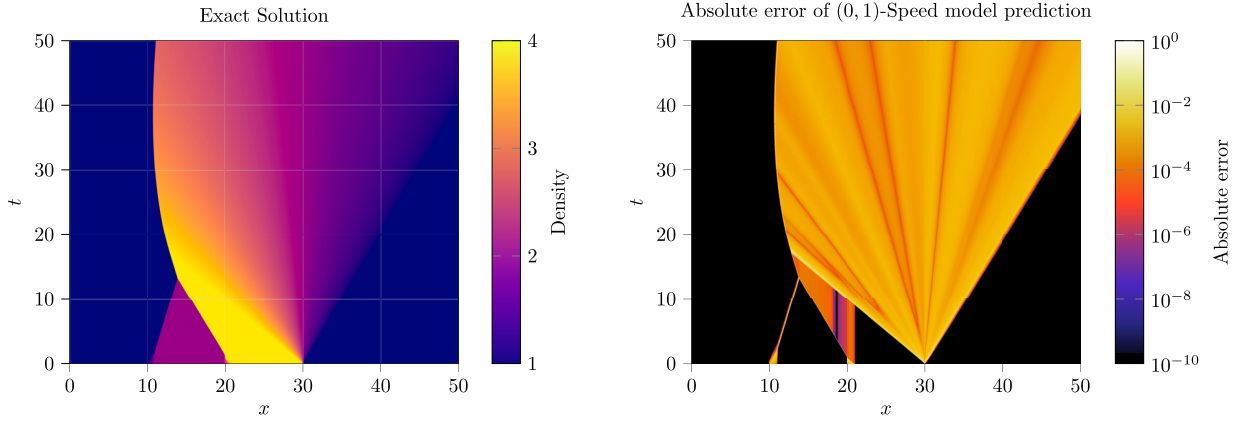


Fig. 9. Time-space diagram of the exact solution (4) discretized on a 5000×5000 grid (left) and absolute error of the (0,1)-Speed Model prediction (right). Comparison of the prediction with the Godunov scheme at various times can be found Fig. 10.

5.1.2. Evaluation on more test cases

The evaluation methodology presented above is limited to solutions for piecewise-constant initial datum, in space-time grids from 20×20 to 1000×1000 cells. We assume that this test set is representative of the LWR solutions landscape: it features a vast set of rarefactions, shockwaves, and shocks moving in space-time. These are the most challenging cases to assess, and this is what motivated us to train only on Riemann solutions.

We will now extend our evaluation on other solutions that contain features that our test set may lack: namely an even larger time horizon, smooth initial conditions and collapsing wavefronts. We therefore chose two additional solutions on which we evaluate our speed model.

First, we use (4), an explicit solution of Eq. (16) computed with the method of characteristics in [69] that is detailed in Section 2.3. We then discretized it on a 5000×5000 grid with parameters $\delta_x = \delta_t = 0.01$ for $(t, x) \in [0, 50] \times [0, 50]$. This solution, represented in Fig. 9, features several shocks and rarefactions that are commonly seen in solutions of Eq. (16). As long as our schemes produce similar solutions, we will only focus on the speed model for comparison in the following and will compare it with the Godunov scheme described in Section 3.2.2.

First, our scheme still outperforms the Godunov scheme for a 5000×5000 discretized solution when measuring the error in ℓ^2 . The (0,1)-Speed Model ℓ^2 error is actually 50% lower than the error from the Godunov scheme prediction. We now provide several plots that compare the prediction of our schemes and the Godunov scheme. We eventually down-sampled the grid to better visualize the features of the solutions.

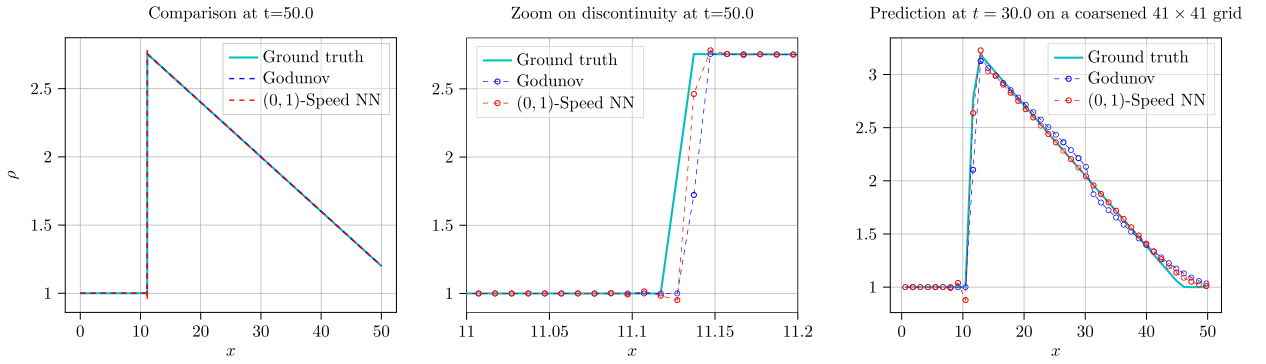


Fig. 10. Comparisons of the last prediction of the (0,1)-speed NN model vs Godunov scheme. The two right plots show predictions on 5000×5000 grids, and the far right one shows the prediction on a coarsened 41×41 grid.

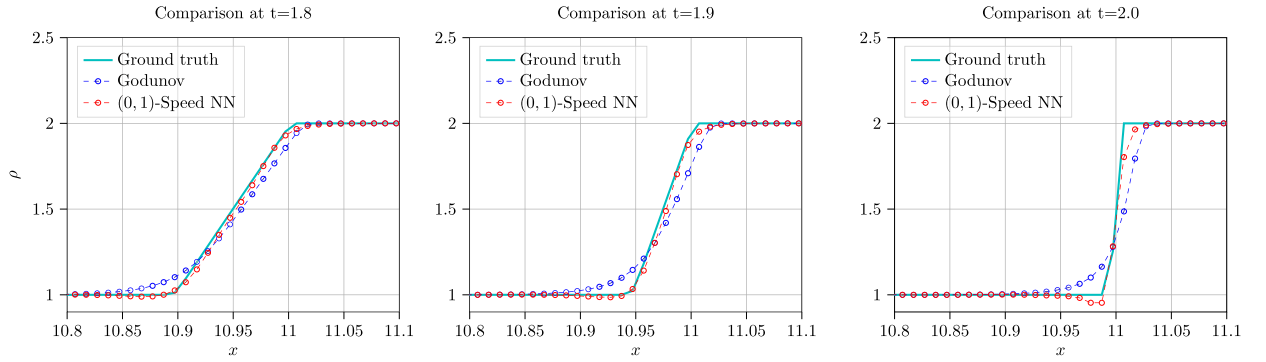


Fig. 11. Successive predictions of the Speed Model and Godunov on the inverse slope collapse ($t = 2.0$) observed in our explicit solution (4). One can note that the Speed model was only trained on solutions of Riemann problem, and generalizes very nicely to this new structure in the solution.

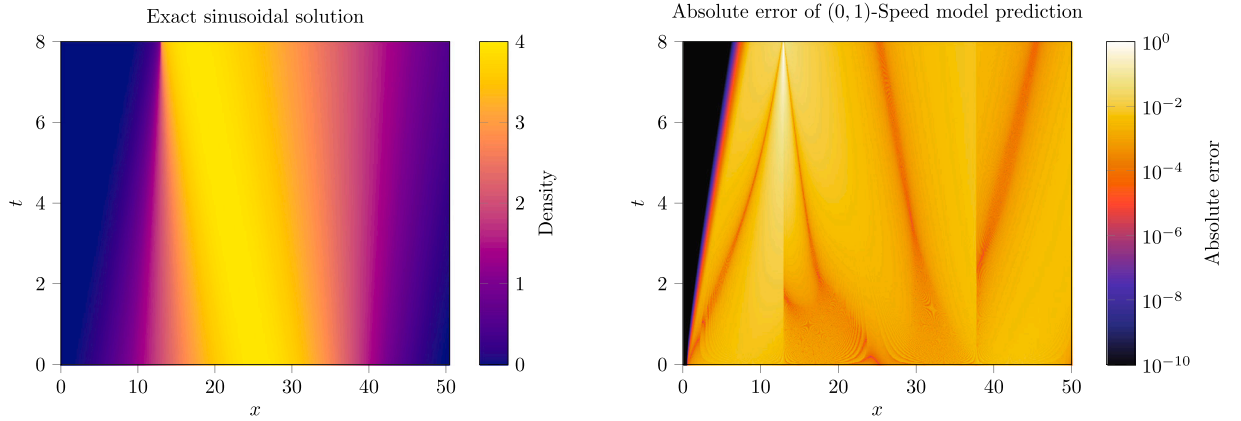


Fig. 12. Time-space diagram of the solution of Eq. (16) for sinusoidal initial datum (Left, Computed to machine precision with Lax-Hopf (2.2)). And absolute error of the prediction from the (0,1)-Speed NN model (right).

Let us now verify that our schemes also generalize well to solutions for smooth initial datum. To that extent, we compute the solution of Eq. (16) for the following sinusoidal initial condition:

$$\forall x \in [0, L], \quad \rho_0(x) = 2(1 - \cos(2\pi x/L))$$

The corresponding solution develops a discontinuity at $t = 8$. For our numerical discretization, we chose a space-time grid of 1262×800 cells of size $\delta x = 0.04, \delta t = 0.01$ (CFL of $1/4$), so that the solution remains smooth on the considered time horizon. Fig. 12 shows the time-space plot for the solution for $t \in [0, 8]$, along with the absolute error of the trained (0,1)-Speed NN model. Fig. 13 further illustrates a comparison between the predictions generated by the Godunov scheme and those produced by our model.

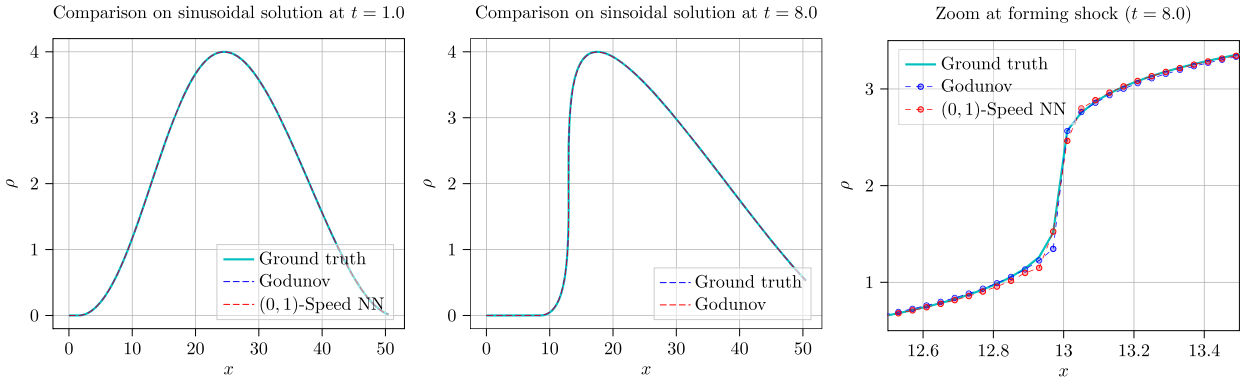


Fig. 13. Comparison of prediction of the (0,1)-speed NN model vs Godunov scheme. The two right plots show predictions of smooth sinusoidal solution at first time-steps (left) and at the last one (800th time-step, $t = 8.0$), and the far right one shows a zoom on the forming shock in the solution.

Table 1

Summary of all benchmarks for LWR-Greenshield (16) solutions in this contribution. The error is always computed as the ℓ^2 -norm of the difference between ground truth and the scheme prediction as defined in Eq. (23). For the two left columns, we provide the reader with the mean ℓ^2 -error over all the discretizations considered in Fig. 8. In cases of the Explicit solution Eq. (4) and the Sinusoidal solution, we show the ℓ^2 -error.

	Benchmark Scheme	TestSet (CFL=1) Avg. ℓ^2 over all disc.	TestSet (CFL=1/2) Avg. ℓ^2 over all disc.	Explicit solution Eq. (4) ℓ^2 -error	Sinusoidal solution ℓ^2 -error
Baselines	Lax-Friedrichs	6.5	/	1.44	/
	Engquist-Osher	2.5	3.1	0.77	0.079
	Godunov	2.5	3.1	0.82	0.086
Ours	Polynomial	2.4	3.1	0.83	0.107
	(0,1)-Flow NN	1.6	2.3	0.42	0.058
	(0,1)-Speed NN	1.6	2.3	0.41	0.049

The error of the (0,1)-speed NN model is 43% lower than that of Godunov scheme when measuring in ℓ^2 (CFL = 1/4). Even-though it was trained only on Riemann problems with even discretizations (CFL=1), our scheme generalizes well to smooth solutions, it once again accurately approximates the wavefront collapse at $t = 8$ and the emergence of the shock-wave. (See Table 1.)

5.2. Numerical schemes analysis

The chosen structure of our models allow the extraction of quantitative knowledge from the learned numerical flux. Let us now explore in depth what the model learned from its training on Riemann problems.

5.2.1. Learned flux function

If the solution is constant, the discretized FVM in Eq. (7) is then equivalent to its continuous version in Eq. (1), and the numerical flux should then be equal to the real underlying flux. Thus, for any (a, b) -stencil numerical function $F : \mathbb{R}^{a+b+1} \rightarrow \mathbb{R}$ approximating the numerical flux $F_{i+1/2}^n$ we should have

$$F(\rho, \dots, \rho, \dots, \rho) = f(\rho).$$

We can therefore extract the learned flux function from the learned numerical fluxes, the results are depicted in Fig. 14.

We see here that the speed model approximates the Greenshield flux to high precision. However, the Flow model seems to be off by a constant. This was expected, considering the fact that the problem of finding an optimal numerical flux is ill-posed: There is no unique optimal numerical flux function for the conservative scheme in Eq. (7). If an optima exists, it is defined modulo a constant shift. We can fix this by extracting the learned flux function $\tilde{f}$ shifted by a constant to get $\tilde{f}(0) = 0$, which gives

$$\begin{aligned} \tilde{f} : \mathbb{R} &\rightarrow \mathbb{R} \\ \rho &\mapsto F_p(\rho, \rho) - F_p(0, 0). \end{aligned}$$

With this, the flow model has indeed also approximated the underlying flux function in the data to high precision as shown in the rightmost graph of Fig. 14.

5.2.2. Fingerprints of learned numerical fluxes

From here, we only focused on (0,1)-stencils numerical flux functions $(\rho_i^n, \rho_{i+1}^n) \mapsto F(\rho_i^n, \rho_{i+1}^n) \approx F_{i+1/2}^n$ that define (1,1)-stencil schemes in conservation form Eq. (7), as the one used in Godunov and Engquist-Osher schemes. These 2-dimensional functions

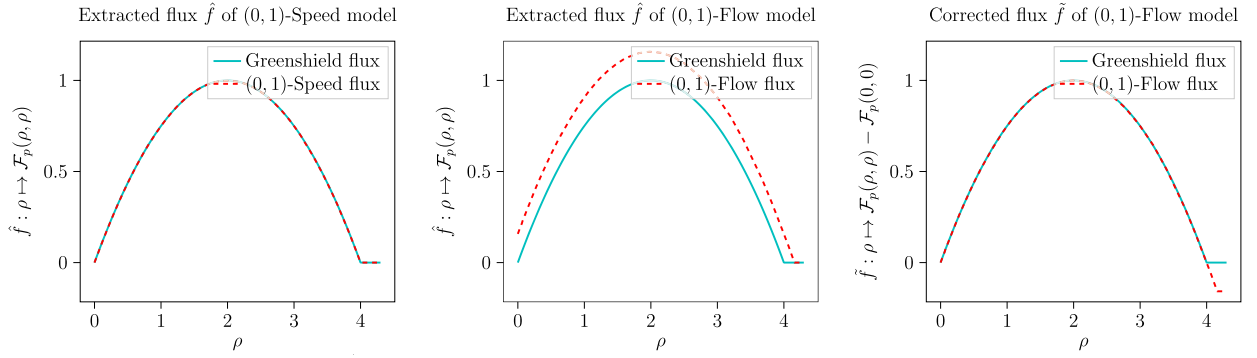


Fig. 14. Learned flux functions $\hat{f} : \rho \mapsto \mathcal{F}_p(\rho, \rho)$ for models trained on Riemann solution for the LWR-Greenshield PDE. We show some values above $\rho_{\max}$ to stress the fact that the ReLU activation enforces non negative flux values.

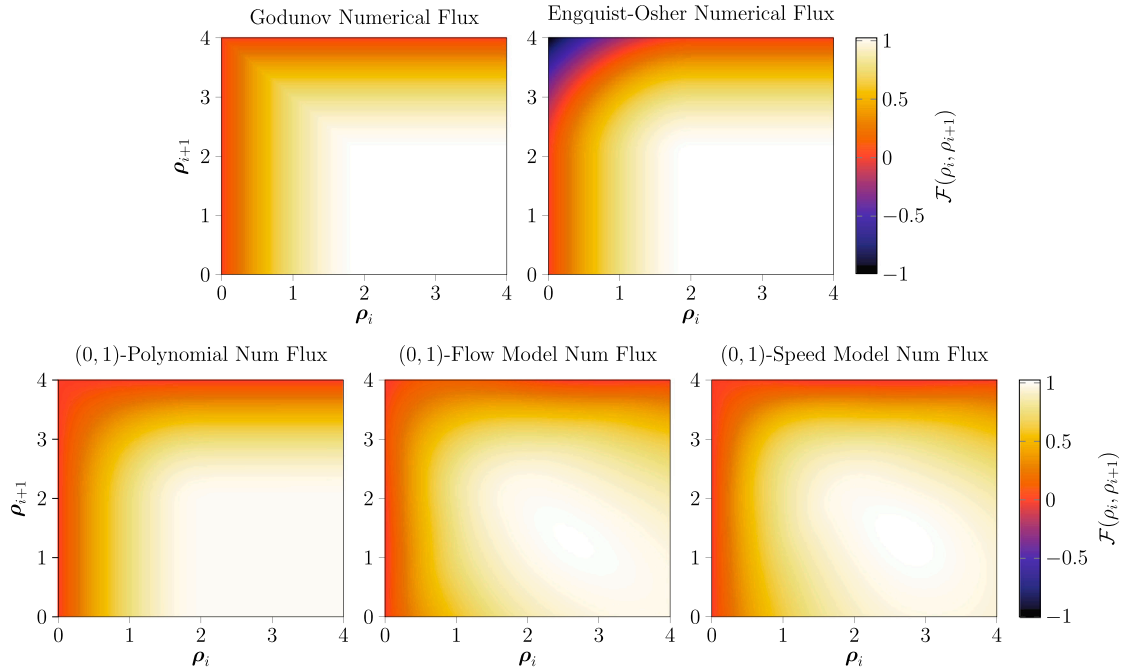


Fig. 15. Fingerprints of known schemes (up) and of learned models (bottom). Note that the fingerprint of the (0,1)-Flow model has been shifted by a constant for easier comparison.

allow straightforward visualization, we therefore compare the resulting “fingerprints” in Fig. 15. Those plots show the output of the numerical flux $\mathcal{F}$ for any input in $[0, \rho_{\max}]^2$. It is noticeable that the models approximating speed Eq. (13) and flow Eq. (11) converged to almost the same numerical flux. This strongly suggests that we have found some kind of optima for the objective defined in Section 4.2.2.

5.2.3. Extension to other flux functions

We also trained our models on data generated with a triangular flux function, meaning a piecewise linear function as stated in Remark 1. Again, we trained the models on a representative set of discretized Riemann solutions and obtained numerical fluxes that learned the underlying features of the PDE. The learned flux function is presented in Fig. 16.

We also extended our study to the case of Underwood’s velocity (see Remark 1 and [51]). The conservation law subject to this velocity also features shocks and rarefaction waves, the rarefactions are however more complex as they are no longer linear. We trained a (0,1)-Speed NN following the very same procedure as in Section 4.2.1, and using the same distribution of computed Riemann solutions as a training set. This proved to work just as well as with Greenshields data. Fig. 17 displays the learned flux function, and Fig. 18 showcases the behavior of the approximated solution at time $t \in \{1, 12\}$, compared to the Godunov baseline and the ground truth.

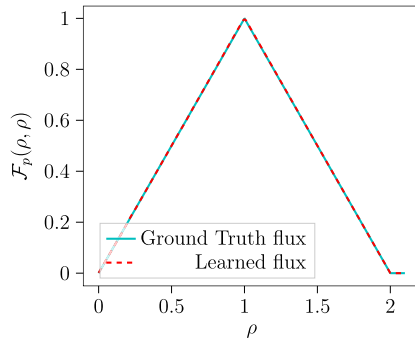


Fig. 16. Extracted flux of a (0, 1)-Speed NN Model trained on explicit Riemann solutions of the LWR model with triangular flux.

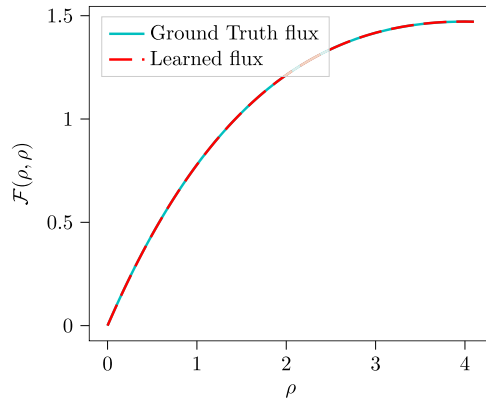


Fig. 17. Extracted flux of a (0, 1)-Speed NN Model trained on explicit Riemann solutions of the LWR model with the Underwood flux. $f(\rho) = v_m \rho e^{-\rho/\rho_{\max}}$.

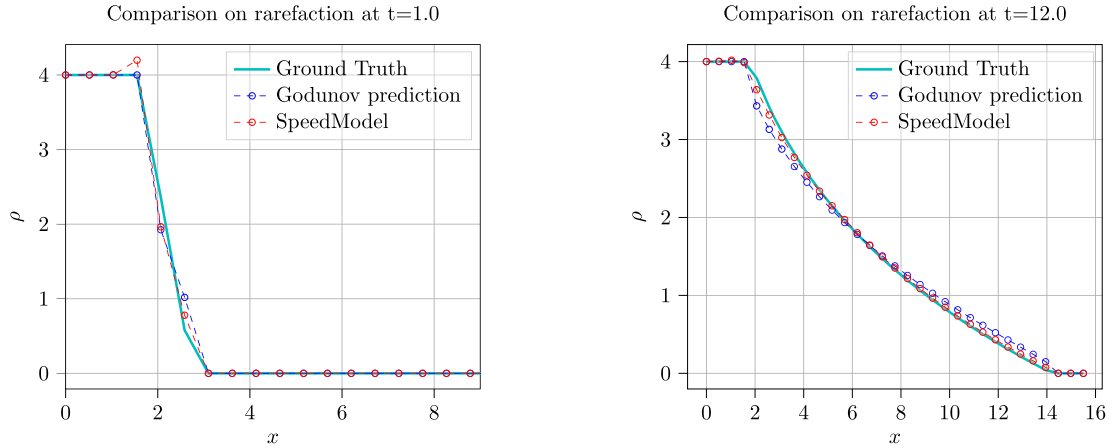


Fig. 18. Comparison of the prediction of the (0, 1)-Speed NN Model versus the Godunov scheme on a rarefaction solution (grid of 31 cells with $\delta x = 0.5$) for the LWR model with Underwood flux. We plot the predicted solution at early stage ($t = 1 = 2\delta t$) where our model predicts a peak at the discontinuity, and also at a later state ($t = 12 = 24\delta t$) where it clearly better fits the exponential curve of the solution than the Godunov scheme.

5.2.4. Total variation diminishing

Given a discretized solution $\rho \in \mathbb{R}^{N_x \times (N_t - 1)}$, the *Total Variation* at time t_n is computed as

$$TV(\rho^n) = \sum_{i=1}^{N_x-1} |\rho_{i+1}^n - \rho_i^n|.$$

Definition 5.1. A numerical scheme is said to be Total Variation Diminishing (TVD) if and only if

$$\forall n \in \{0, \dots, N_t - 1\} \quad TV(\rho^{n+1}) \leq TV(\rho^n).$$

In the case of scalar conservation laws, the TVD property ensures that the scheme will not produce non-physical solutions by forbidding spurious oscillations. The learned schemes are unfortunately not TVD, which is visible in Fig. 10 and Fig. 11: one can see the small spurious oscillations that appear at the shock waves. This behavior is commonly seen in non-TVD or higher-order schemes and is therefore not considered a problem of the NN models.

Monotonicity preserving Monotonicity can be a very important property for first-order schemes. It has indeed been proven ([70, Harten, 1983]) that monotone schemes are TVD.

Definition 5.2 (Monotone scheme). For a conservative scheme H (see Eq. (10)) with a $(0, 1)$ -stencil numerical flux function

$$H(\rho_{i-1}, \rho_i, \rho_{i+1}) = \rho_i - \frac{\delta_t}{\delta_x} (F(\rho_i, \rho_{i+1}) - F(\rho_{i-1}, \rho_i)) \quad (26)$$

then H is monotone if and only if

$$\frac{\partial F}{\partial \rho_1} \geq 0 \wedge \frac{\partial F}{\partial \rho_2} \leq 0 \wedge \frac{\partial F}{\partial \rho_1} - \frac{\partial F}{\partial \rho_2} \leq \frac{\delta_x}{\delta_t}. \quad (27)$$

One can see looking at Fig. 15 that the optimized NN numerical fluxes function F do not fulfill the conditions of Eq. (27) numerically. The resulting schemes are therefore not monotone. This should not be surprising: the *Godunov Theorem* (originally proved in [71]) states that monotone schemes can be at most first-order accurate. Non-monotone schemes, that are able to generate new extrema, can *a priori* better approximate solutions, at the price of risking to compute non-physical solutions. To enforce a monotonicity property on the schemes generated in this work, one has to either restrict the function space or constrain the parametrization considered in the optimization problem of Eq. (18). As the constraining of the parametrization of neural networks or polynomials results in rather complicated boundaries to traverse for iterative optimization methods, we constructed a simplistic inherently monotone scheme (for large enough δ_x/δ_t) as described in Section 3.4. The method fulfills by design the monotonicity conditions Eq. (27) for large enough δ_x/δ_t .

5.2.5. Extension to wider stencils

The general framework for (a, b) -NN models in Definition 3.1 developed in Section 3.3.3 allows for the use of any input stencil size to approximate the numerical flux: this is motivated by the will to apply those models to higher-order PDEs and to real-world data. The LWR-Greenshield PDE is of first-order in space, the flux at $x_{i+1/2}$ depends only on the local value of ρ . In the discrete FVM framework, it is reasonable to think that the numerical Flux $F_{i+1/2}^n$ should also depend only on local values of ρ . This suggests the choice of a $(0, 1)$ -stencil numerical flux approximation between cells ρ_i^n and ρ_{i+1}^n : $F_{i+1/2}^n \approx F(\rho_i^n, \rho_{i+1}^n)$. This together with the well-known fact that solutions to nonlinear conservation laws are not smooth and even for smooth initial datum might develop discontinuities in finite time. Thus, typical Taylor approximations or higher-order quadrature formulae might fail. We here extend the previous work to bigger stencils in order to validate this idea and the capacity of our models to learn with wider stencils.

We performed the same training routine as the previous $(0, 1)$ -NN models to train a $(1, 2)$ -speed model with the same architecture $d = 6, h = 15$, i.e. 1291 parameters. The latter expectedly performs better than the other models as it has access to more information. We however notice that it seemed to converge to almost the same scheme as the previous $(0, 1)$ -stencil ones. We provide the convergence plot including the $(1, 2)$ -speed model Fig. 19.

We also illustrate the 2-dimensional fingerprint of both $(0, 1)$ - and $(1, 2)$ -speed models in Fig. 20. Note that if it describes completely the behavior of the $(0, 1)$ -speed model, it doesn't show the dependency of the $(1, 2)$ -speed model on its two other variables.

The striking similarity between the observed fingerprints suggests that the $(1, 2)$ -speed models use almost only the two middle points of its $(1, 2)$ -stencil to compute its speed estimate, which would be a strong argument in favor of what was said above. In order to quantify this dependency, we computed the Pearson correlations between the input stencil and the output speed prediction. In the Greenshield case (Eq. (16)), the speed is a linear function of the density: $v(\rho) = v_{\max} \left(1 - \frac{\rho}{\rho_{\max}}\right)$, so the linear dependency assessment to compute the variance and the Pearson coefficients is reasonable. One can also note that in the continuous case, there is a perfect negative correlation of -1 between $v(\rho)$ and ρ . The results for our $(0, 1)$ and $(1, 2)$ -stencil speed functions are shown in Fig. 21.

This backs up our previous statement, suggesting that the $(1, 2)$ -speed model uses almost only ρ_i^n and ρ_{i+1}^n to compute $\hat{v}_i^n$. This justifies the choice of Godunov and other 3-point stencil schemes using $(0, 1)$ -stencil numerical flux functions. The two additional points given to the $(1, 2)$ -speed model, even if not strongly correlated to the prediction, still give enough information to the scheme to significantly improve the performance of the model, as shown in Fig. 19.

6. Conclusions and next steps

With no knowledge about the PDE except the conservation of mass, our models have been able to learn the underlying physics of the simple LWR equation with concave fluxes. We have even found first-order schemes that perform better, in the ℓ^2 -norm, than

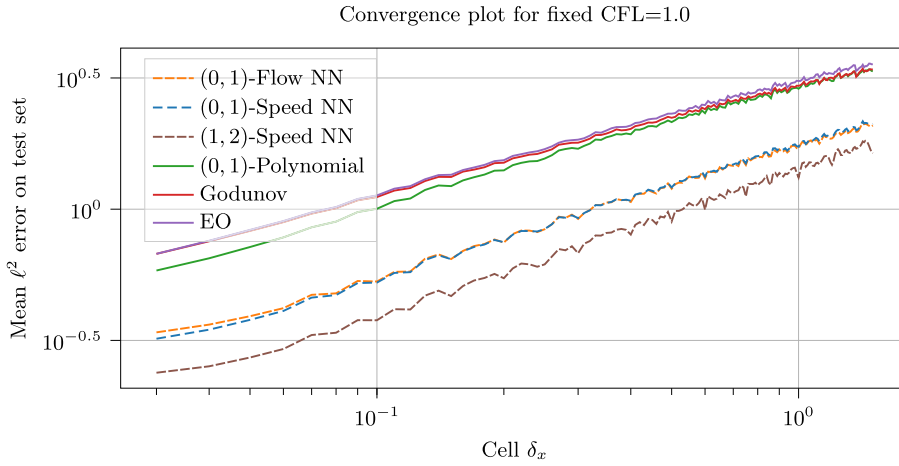


Fig. 19. Convergence Plot including the (1,2)-speed model.

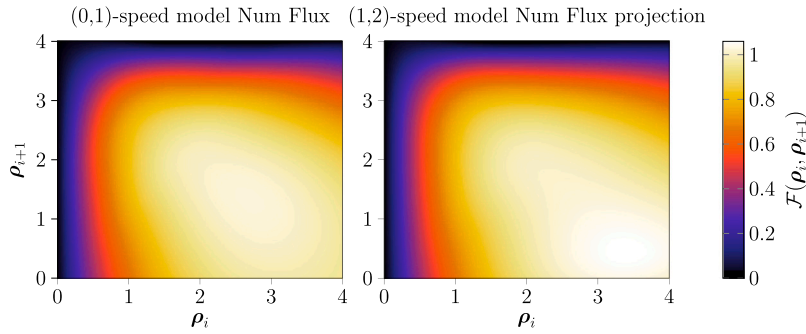
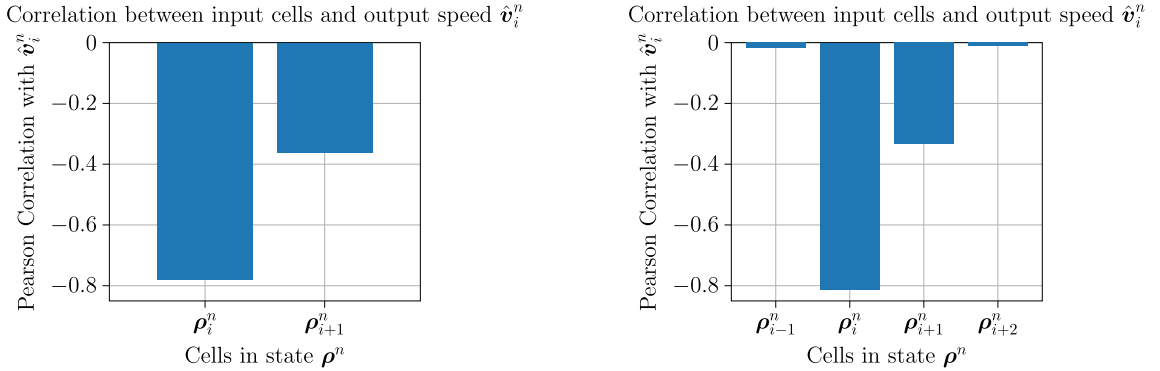
Fig. 20. Fingerprints of the trained (0,1)-speed model and (1,2)-speed model. Note that the (1,2)-speed model numerical flux is a 4-dimensional scalar function, only its behavior on two points is here shown for comparison. The left figure then shows only the projection function $(\rho_i, \rho_{i+1}) \mapsto \mathcal{F}(\rho_i, \rho_i, \rho_{i+1}, \rho_{i+1})$.

Fig. 21. Comparisons of the correlations between input cells and output speed of the (0,1) and (1,2)-Speed NN models.

historical schemes that use exact knowledge about the flux function relation on a wide range of numerical discretizations and for long time horizons.

6.1. Summary of methods and tradeoffs

Throughout this work, we used neural networks to learn an optimal numerical flux function for Eq. (7). We mostly worked with simple functions $\mathcal{F} \in C^0(\mathbb{R}^2 \rightarrow \mathbb{R})$ and one could argue that we are using a sledgehammer to crack a nut. The NNs that we used are indeed quite sophisticated compared to the simplicity of the Godunov scheme in Section 3.2.2 or the piecewise polynomial flux described in Section 3.4. There are however good reasons to choose neural networks as numerical flux functions. First, with no insight into the potential complexity of an optimal numerical flux, neural networks were, as universal approximators, obviously a

good candidate to try. Moreover, by their repetitive nature combining affine and non-linear functions, neural networks represent at first glance more complicated structure than functions built from algebraic primitives. Second, Neural networks are by nature very fluid when it comes to changing their architecture: This flexibility enables us to easily change the stencil size for instance, as shown in Section 5.2.5. This will be useful if one wants to apply our methods to other problems that would need wider stencils. Finally, the optimization problem described in Section 4.2 is computationally intensive and we heavily benefited from the tremendous resources invested in the optimization of deep learning software frameworks that have resulted from the explosion of the deep learning community. In particular, the *backpropagation through time* algorithm is a commonly used algorithm encountered in the RNN field, and a lot of implementations are readily available. Regarding the quality of the obtained solutions, one should note that reaching a global minimum for the presented class of optimization problems is known to be practically impossible. While there will likely be similar or slightly better solution candidates, the fact that we obtain almost the same scheme with wider stencils, albeit with a different representation, may suggest that we found a good candidate.

The piecewise polynomials of Section 3.4 present objects that can easily be globally analyzed due to their simple algebraic structure. As the parameters are interpolation points, the parametrization does not seem to induce complicated associated optimization problems. Being a product of one-dimensional functions, the function class generalizes to higher dimensional domains, i.e., larger stencil sizes. Although not considered in this work, by summing functions of the type described in Section 3.4 one may approximate more complicated functions.

6.2. Future steps and extensions

In light of our results, the following directions look promising.

Field operations. The data used in this preliminary work can be considered as a benchmark to test our methods against, knowing exactly what we should obtain, especially the underlying PDEs. Now confident that our method can learn those physical laws from very simple data, we can now apply our methods to real-world data with good hope that they will learn insightful knowledge about the mysterious physics of traffic. Future work will explore the learning of numerical solvers on real available density data such as the [Caltrans Performance Measurement System](#) (PeMS), or density data extracted from video footage of highways [72,73].

A thorough runtime analysis. We did not explore the time and CPU efficiency of the considered methods in depth. The main reason is that all are first-order numerical schemes, and they are all by nature very efficient to compute with linear complexity in the number of cells. The neural network models are even so far more complex, and thus slower to compute predictions than explicit schemes such as Godunov or Engquist-Osher (see Section 3) or than the simple polynomial functions described in Section 3.4. Even so, once the model is trained, we are left with an (a, b) -stencil function that can then be embedded into faster implementations. This is possible by compressing it in smaller NN—this compression method is called *distillation* (see [74]) in the machine learning community—but it is also possible to model it with piecewise polynomials or even—at a memory cost—to store a sufficiently dense map of the model outputs and to perform interpolation for instance.

A full comparison with WFT. Wave-front tracking is a discretization method based on tracking a finite number of wavefronts generated by solving Riemann problems and approximating rarefactions with rarefaction shocks of small strength (see [2]). The latter are non-entropic, but in the limit, the entropy condition is recovered. The method is semidiscrete since the time evolution of fronts is continuous. Because of the different nature of the WFT scheme from those described here, a full comparison is challenging and will be performed in future work.

Optimizing on an adaptive time-stepping: The methods to learn the numerical flux have for now been considered on a fixed time-grid. However, one could keep the time-discretization variable and optimize additionally for the time-stepping. This would hopefully lead to a scheme which can decide a priori only based on the current state whether it should decrease or increase the time-step. As the smaller the time stepping the more precise the solution, this would lead to an ill-posed problem which is the reason why one would need to specify in addition a maximal number of time-steps on which the algorithm would choose the best distribution.

Control. In recent years, deep reinforcement learning (RL) has gained significant attention as an emerging paradigm for the development of robust, high-quality controllers across a spectrum of domains. With its ability to learn complex behaviors and decision-making processes, deep RL has demonstrated remarkable success in fields such as robotics [75,76], surpassing human experts in games [77–79], or traffic control [80], and has transformative potential across a wide range of applications. In particular, [81] provides an ML approach for state estimation, [82,83] investigates the application of functional PDE coefficients for controllers and in particular backstepping. [84] addresses similar questions, this time incorporating the natural and application-driven delay on the state of the system and compensating for such a delay via machine learning.

CRedit authorship contribution statement

Victor Morand: Conceptualization, Methodology, Project administration, Resources, Data curation, Formal analysis, Investigation, Software, Validation, Visualization, Writing – original draft, Writing – review & editing. **Nils Müller:** Writing – review & editing, Conceptualization, Formal analysis, Investigation, Methodology, Project administration, Resources, Software, Validation, Visualization, Writing – original draft. **Ryan Weightman:** Data curation, Investigation, Methodology, Resources, Writing – original draft, Writing – review & editing. **Benedetto Piccoli:** Conceptualization, Formal analysis, Methodology, Supervision, Validation,

Writing – original draft. **Alexander Keimer:** Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing. **Alexandre M. Bayen:** Supervision, Validation, Writing – original draft, Conceptualization, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgements

We would like to thank the anonymous referees for their suggestions to improve the paper's readability and content.

References

- [1] L.C. Evans, *Partial Differential Equations*, 2nd edition, Graduate Studies in Mathematics, vol. 19, American Mathematical Society, Providence, RI, 2010.
- [2] A. Bressan, *Hyperbolic Systems of Conservation Laws: The One-Dimensional Cauchy Problem*, Oxford Lecture Series in Mathematics and Its Applications, vol. 20, Oxford University Press, Oxford, 2000.
- [3] J. Glimm, P.D. Lax, *Decay of Solutions of Systems of Nonlinear Hyperbolic Conservation Laws*, vol. 101, American Mathematical Soc., 1970.
- [4] E. Godlewski, P.-A. Raviart, *Numerical Approximation of Hyperbolic Systems of Conservation Laws*, vol. 118, Springer Science & Business Media, 2013.
- [5] P.G. LeFloch, *Hyperbolic Systems of Conservation Laws: The Theory of Classical and Nonclassical Shock Waves*, Springer Science & Business Media, 2002.
- [6] M. Garavello, K. Han, B. Piccoli, et al., *Models for Vehicular Traffic on Networks*, vol. 9, American Institute of Mathematical Sciences (AIMS), Springfield, MO, 2016.
- [7] O. Oleinik, Discontinuous solutions of non-linear differential equations, *Usp. Mat. Nauk* 12 (1957) 3–73.
- [8] M. Aizerman, E. Bredihina, S. Černikov, F. Gantmaher, I. Gelfand, S. Gelfer, D. Harazov, M. Kadec, J. Korobeĭnik, M. Kreĭn, O. Oleĭnik, I. Pyateckii-Šapiro, M. Subhankulov, K. Temko, A. Tureckii, *Seventeen Papers on Analysis*, American Mathematical Society, 1963.
- [9] R.J. LeVeque, *Finite Volume Methods for Hyperbolic Problems*, Cambridge Texts in Applied Mathematics, Cambridge University Press, 2002.
- [10] H. Holden, N.H. Risebro, *Front Tracking for Hyperbolic Conservation Laws*, vol. 152, Springer, 2015.
- [11] P.K. Sweby, High resolution schemes using flux limiters for hyperbolic conservation laws, *SIAM J. Numer. Anal.* 21 (5) (1984) 995–1011, <https://doi.org/10.1137/0721062>.
- [12] D.I. Ketcheson, S. Gottlieb, C.B. Macdonald, Strong stability preserving two-step Runge–Kutta methods, *SIAM J. Numer. Anal.* 49 (6) (2011) 2618–2639, <https://doi.org/10.1137/10080960X>.
- [13] G.-S. Jiang, C.-W. Shu, Efficient implementation of weighted eno schemes, *J. Comput. Phys.* 126 (1) (1996) 202–228, <https://doi.org/10.1006/jcph.1996.0130>, <https://www.sciencedirect.com/science/article/pii/S0021999196901308>.
- [14] X.-D. Liu, S. Osher, T. Chan, Weighted essentially non-oscillatory schemes, *J. Comput. Phys.* 115 (1) (1994) 200–212, <https://doi.org/10.1006/jcph.1994.1187>, <https://www.sciencedirect.com/science/article/pii/S0021999184711879>.
- [15] S. Clain, S. Diot, R. Loubère, A high-order finite volume method for systems of conservation laws—multi-dimensional optimal order detection (mood), *J. Comput. Phys.* 230 (10) (2011) 4028–4050, <https://doi.org/10.1016/j.jcp.2011.02.026>.
- [16] P. Lasaint, P. Raviart, On a finite element method for solving the neutron transport equation, in: C. de Boor (Ed.), *Mathematical Aspects of Finite Elements in Partial Differential Equations*, Academic Press, 1974, pp. 89–123, <https://www.sciencedirect.com/science/article/pii/B978012208350150008X>.
- [17] *Discontinuous Galerkin Method for Conservation Laws*, Birkhäuser Basel, Basel, 2009, pp. 157–174.
- [18] K. Hornik, M. Stinchcombe, H. White, Multilayer feedforward networks are universal approximators, *Neural Netw.* 2 (5) (1989) 359–366.
- [19] A.G. Baydin, B.A. Pearlmutter, A.A. Radul, J.M. Siskind, Automatic differentiation in machine learning: a survey, *J. Mach. Learn. Res.* 18 (2018) 1–43.
- [20] M. Raissi, P. Perdikaris, G. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707, <https://doi.org/10.1016/j.jcp.2018.10.045>, <https://www.sciencedirect.com/science/article/pii/S0021999118307125>.
- [21] X. Di, R. Shi, Z. Mo, Y. Fu, Physics-informed deep learning for traffic state estimation: a survey and the outlook, *Algorithms* 16 (6) (2023) 305.
- [22] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural Comput.* 9 (1997) 1735–1780, <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [23] P. Hoedt, F. Kratzert, D. Klotz, C. Halmich, M. Holzleitner, G. Nearing, S. Hochreiter, G. Klambauer, MC-LSTM: mass-conserving LSTM, CoRR, arXiv:2101.05186 [abs], 2021.
- [24] B. Stevens, T. Colonius, Finitenet: a fully convolutional LSTM network architecture for time-dependent partial differential equations, CoRR, arXiv:2002.03014 [abs], 2020.
- [25] S. Mishra, R. Molinaro, Estimates on the generalization error of physics-informed neural networks for approximating PDEs, *IMA J. Numer. Anal.* 43 (1) (2022) 1–43, <https://doi.org/10.1093/imanum/drab093>, <https://academic.oup.com/imajna/article-pdf/43/1/1/49059512/drab093.pdf>.
- [26] T.D. Ryck, S. Mishra, R. Molinaro, Wpinns: weak physics informed neural networks for approximating entropy solutions of hyperbolic conservation laws, arXiv:2207.08483, 2022.
- [27] F. Chen, J. Huang, C. Wang, H. Yang, Friedrichs learning: weak solutions of partial differential equations via deep learning, *SIAM J. Sci. Comput.* 45 (3) (2023) A1271–A1299.
- [28] T. De Ryck, S. Mishra, Error analysis for deep neural network approximations of parametric hyperbolic conservation laws, 2022.
- [29] R. Arora, A deep learning framework for solving hyperbolic partial differential equations: part I, preprint, arXiv:2307.04121, 2023.
- [30] B. Stevens, T. Colonius, Enhancement of shock-capturing methods via machine learning, *Theor. Comput. Fluid Dyn.* 34 (4) (2020) 483–496, <https://doi.org/10.1007/s00162-020-00531-1>.
- [31] J. Guo, Y. Yao, H. Wang, T. Gu, Pre-training strategy for solving evolution equations based on physics-informed neural networks, *J. Comput. Phys.* 489 (2023) 112258.
- [32] E.L. Strelow, A. Gerisch, J. Lang, M.E. Pfetsch, Physics informed neural networks: a case study for gas transport problems, *J. Comput. Phys.* 481 (2023) 112041.

- [33] A. Bihlo, R.O. Popovych, Physics-informed neural networks for the shallow-water equations on the sphere, *J. Comput. Phys.* 456 (2022) 111024.
- [34] Exact Conservation Laws for Neural Network Integrators of Dynamical Systems, *J. Comput. Phys.* 488 (2023) 112234.
- [35] R.G. Patel, I. Manickam, N.A. Trask, M.A. Wood, M. Lee, I. Tomas, E.C. Cyr, Thermodynamically consistent physics-informed neural networks for hyperbolic systems, *J. Comput. Phys.* 449 (2022) 110754, <https://doi.org/10.1016/j.jcp.2021.110754>.
- [36] J. Magiera, D. Ray, J.S. Hesthaven, C. Rohde, Constraint-aware neural networks for Riemann problems, *J. Comput. Phys.* 409 (2020) 109345, <https://doi.org/10.1016/j.jcp.2020.109345>.
- [37] Z.-Y. Zhang, H. Zhang, L.-S. Zhang, L.-L. Guo, Enforcing continuous symmetries in physics-informed neural network for solving forward and inverse problems of partial differential equations, *J. Comput. Phys.* 492 (2023) 112415.
- [38] H. Huang, Y. Liu, V. Yang, Neural networks with inputs based on domain of dependence and a converging sequence for solving conservation laws, part I: 1d Riemann problems, arXiv:2109.09316, 2021.
- [39] H. Huang, Y. Liu, V. Yang, Neural networks with local converging inputs (nnlci) for solving conservation laws, part ii: 2d problems, arXiv:2204.10424, 2022.
- [40] N. Thurety, P. Holl, M. Mueller, P. Schnell, F. Trost, K. Um, Physics-based Deep Learning, <https://physicsbaseddeeplearning.org>, 2021.
- [41] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, A. Anandkumar, Neural operator: learning maps between function spaces, preprint, arXiv:2108.08481, 2021.
- [42] N. Nguyen-Fotiadis, M. McKerns, A. Sornborger, Machine learning changes the rules for flux limiters, *Phys. Fluids* 34 (8) (Aug. 2022), <https://doi.org/10.1063/5.0102939>.
- [43] J. Yu, J.S. Hesthaven, A data-driven shock capturing approach for discontinuous Galekin methods, *Comput. Fluids* 245 (2022) 105592, <https://doi.org/10.1016/j.compfluid.2022.105592>, <https://www.sciencedirect.com/science/article/pii/S0045793022002006>.
- [44] S. Mishra, A machine learning framework for data driven acceleration of computations of differential equations, arXiv:1807.09519, 2018.
- [45] L. Bois, E. Franck, L. Navoret, V. Vigon, An optimal control deep learning method to design artificial viscosities for discontinuous Galerkin schemes (2023), <https://doi.org/10.48550/ARXIV.2309.11795>, <https://arxiv.org/abs/2309.11795>.
- [46] Y. Bar-Sinai, S. Hoyer, J. Hickey, M.P. Brenner, Learning data-driven discretizations for partial differential equations, *Proc. Natl. Acad. Sci.* 116 (31) (2019) 15344–15349.
- [47] J. Zhuang, D. Kochkov, Y. Bar-Sinai, M.P. Brenner, S. Hoyer, Learned discretizations for passive scalar advection in a two-dimensional turbulent flow, *Phys. Rev. Fluids* 6 (2021) 064605.
- [48] G. Dresdner, D. Kochkov, P.C. Norgaard, L. Zepeda-Nunez, J. Smith, M. Brenner, S. Hoyer, Learning to correct spectral methods for simulating turbulent flows, *Transact. Mach. Learn. Res.* (2023).
- [49] D. Ray, J.S. Hesthaven, An artificial neural network as a troubled-cell indicator, *J. Comput. Phys.* 367 (2018) 166–191.
- [50] A.D. Beck, J. Zeifang, A. Schwarz, D.G. Flad, A neural network based shock detection and localization approach for discontinuous Galerkin methods, *J. Comput. Phys.* 423 (2020) 109824.
- [51] M. Garavello, B. Piccoli, *Traffic Flow on Networks*, vol. 1, American Institute of Mathematical Sciences (AIMS), Springfield, MO, 2006.
- [52] A.M. Bayen, R.L. Raffard, C.J. Tomlin, Network congestion alleviation using adjoint hybrid control: application to highways, in: *International Workshop on Hybrid Systems: Computation and Control*, Springer, 2004, pp. 95–110.
- [53] B. Greenshields, W. Channing, H. Miller, et al., A study of traffic capacity, in: *Highway Research Board Proceedings*, vol. 1935, National Research Council (USA), Highway Research Board, 1935.
- [54] S.K. Godunov, I. Bohachevsky, Finite difference method for numerical computation of discontinuous solutions of the equations of fluid dynamics, *Mat. Sb.* 47(89) (3) (1959) 271–306, <https://hal.science/hal-01620642>.
- [55] B. Engquist, S. Osher, One-sided difference approximations for nonlinear conservation laws, *Math. Comput.* 36 (154) (1981) 321–351.
- [56] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, PyTorch: an imperative style, high-performance deep learning library, in: H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, R. Garnett (Eds.), *Advances in Neural Information Processing Systems*, vol. 32, Curran Associates, Inc., 2019, pp. 8024–8035, <http://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.
- [57] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G.S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, R. Jozefowicz, Y. Jia, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, M. Schuster, R. Monga, S. Moore, D. Murray, C. Olah, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, X. Zheng, Tensorflow, large-scale machine learning on heterogeneous systems, <https://doi.org/10.5281/zenodo.4724125>, Nov. 2015.
- [58] J. Bradbury, R. Frostig, P. Hawkins, M.J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, Q. Zhang, JAX: composable transformations of python+NumPy programs, <http://github.com/google/jax>, 2018.
- [59] K. Fukushima, Neocognitron: a self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position, *Biol. Cybern.* 36 (1980) 193–202.
- [60] A.D. May, *Traffic flow fundamentals*, 1990.
- [61] F.N. Fritsch, J. Butland, A method for constructing local monotone piecewise cubic interpolants, *SIAM J. Sci. Stat. Comput.* 5 (2) (1984) 300–304.
- [62] P.J. Werbos, Generalization of backpropagation with application to a recurrent gas market model, *Neural Netw.* 1 (4) (1988) 339–356, [https://doi.org/10.1016/0893-6080\(88\)90007-X](https://doi.org/10.1016/0893-6080(88)90007-X), <https://www.sciencedirect.com/science/article/pii/089360808890007X>.
- [63] M.J. Lighthill, G.B. Whitham, On kinematic waves II. A theory of traffic flow on long crowded roads, *Proc. R. Soc. Lond. Ser. A, Math. Phys. Sci.* 229 (1178) (1955) 317–345.
- [64] P.I. Richards, Shock waves on the highway, *Oper. Res.* 4 (1) (1956) 42–51.
- [65] R.S. Sutton, A.G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018.
- [66] D.P. Kingma, J. Ba, Adam: a method for stochastic optimization, CoRR, arXiv:1412.6980 [abs], 2014.
- [67] R. Pascanu, T. Mikolov, Y. Bengio, On the difficulty of training recurrent neural networks, in: *International Conference on Machine Learning*, Pmlr, 2013, pp. 1310–1318.
- [68] D. Serre, *Systems of Conservation Laws 2: Geometric Structures, Oscillations, and Initial-Boundary Value Problems*, vol. 2, Cambridge University Press, 1999.
- [69] A.M. Bayen, R.L. Raffard, C.J. Tomlin, Network congestion alleviation using adjoint hybrid control: application to highways, in: R. Alur, G.J. Pappas (Eds.), *Hybrid Systems: Computation and Control*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2004, pp. 95–110.
- [70] A. Harten, High resolution schemes for hyperbolic conservation laws, *J. Comput. Phys.* 49 (3) (1983) 357–393, [https://doi.org/10.1016/0021-9991\(83\)90136-5](https://doi.org/10.1016/0021-9991(83)90136-5), <https://www.sciencedirect.com/science/article/pii/0021999183901365>.
- [71] S. Godunov, *Different Methods for Shock Waves* Moscow, State University, 1954.
- [72] M. Ameli, A.M. Bayen, R. Delorenzo, M.L.D. Monache, S. Hornstein, J.W. Lee, M. Mahmood, M. Mahmood, S.T. McQuade, B. Piccoli, B. Seibold, J.M. Sprinkle, D. Timsit, R. Wagner, D. Work, R. Xu, Designing and simulating the 100-car experiment for the circles consortium, *IEEE Control Syst. Mag.* (2023), submitted for publication.
- [73] J.W. Lee, H. Wang, K. Jang, A. Hayat, M. Bunting, A. Alanqary, W. Barbour, Z. Fu, X. Gong, G. Gunter, S. Hornstein, A.R. Kreidieh, N. Lichtlé, M.W. Nice, W.A. Richardson, A. Shah, E. Vinitsky, F. Wu, S. Xiang, S. Almatrudi, F. Althukair, R. Bhadani, J. Carpio, R. Chekroun, E. Cheng, M.T. Chiri, F.-C. Chou, R. Delorenzo, M. Gibson, D. Gloudemans, A. Keimer, N. Khoudari, M. Mahmood, M. Mahmood, H.N.Z. Matin, S. McQuade, R. Ramadan, D. Urieli, Y. Wang, R. Xu, M. Yao, Y. You, G. Zachar, Y. Zhao, S. Bhaskaran, K. Butts, M. Gowda, J. Lee, Z. Zhang, C. Zhou, D.B. Work, B. Seibold, J. Sprinkle, B. Piccoli, M.L.D. Monache, A.M. Bayen, Traffic smoothing via connected & automated vehicles: a modular, hierarchical control design deployed in 100-cav flow smoothing experiment, *IEEE Control Syst. Mag.* (2023), submitted for publication.

- [74] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, preprint, arXiv:1503.02531, 2015.
- [75] A.R. Mahmood, D. Korenkevych, G. Vasan, W. Ma, J. Bergstra, Benchmarking reinforcement learning algorithms on real-world robots, arXiv:1809.07731, 2018.
- [76] S. Gu, E. Holly, T. Lillicrap, S. Levine, Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates, in: 2017 IEEE International Conference on Robotics and Automation (ICRA), 2017, pp. 3389–3396.
- [77] O. Vinyals, I. Babuschkin, W.M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D.H. Choi, R. Powell, T. Ewalds, P. Georgiev, et al., Grandmaster level in starcraft ii using multi-agent reinforcement learning, *Nature* 575 (7782) (2019) 350–354.
- [78] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller, Playing atari with deep reinforcement learning, preprint, arXiv:1312.5602, 2013.
- [79] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, et al., A general reinforcement learning algorithm that masters chess, shogi, and go through self-play, *Science* 362 (6419) (2018) 1140–1144.
- [80] N. Lichtlé, E. Vinitzky, M. Nice, B. Seibold, D. Work, A.M. Bayen, Deploying traffic smoothing cruise controllers learned from trajectory data, in: 2022 International Conference on Robotics and Automation (ICRA), 2022, pp. 2884–2890.
- [81] Y. Shi, Z. Li, H. Yu, D. Steeves, A. Anandkumar, M. Krstic, Machine learning accelerated PDE backstepping observers, in: 2022 IEEE 61st Conference on Decision and Control (CDC), IEEE, 2022.
- [82] M. Krstic, L. Bhan, Y. Shi, Neural operators of backstepping controller and observer gain functions for reaction-diffusion PDEs, 2023.
- [83] L. Bhan, Y. Shi, M. Krstic, Neural operators for bypassing gain and control computations in PDE backstepping, 2023.
- [84] J. Qi, J. Zhang, M. Krstic, Neural operators for delay-compensating control of hyperbolic PIDEs, 2023.